TRABAJO PRÁCTICO OBLIGATORIO

# Programación Orientada a Objetos

## ETAPA 1

Modelado y aplicación de patrones




---

**Proyecto: Sistema de Gestión – Agencia de Turismo Rumbo Sur**


| Integrante | Legajo |
|---|---|
| Bianca Chancalay | 1216027 |
| Guadalupe Olivera | 1223910 |
| Carlos Daniel Lazo | 1244230 |

Profesora: Leon Maria Angela


Materia: Programación Orientada a Objetos

Turno: Tarde · Día de cursada: Jueves · Año: 2026

Versión consolidada con actualización de Etapa 2 · Noviembre 2026

# Índice

1. Análisis del enunciado
2. Requerimientos funcionales
3. Reglas de negocio
4. Supuestos adoptados
5. Casos de uso
6. Diagrama de clases inicial
7. Diagramas de secuencia
8. Aplicación de principios SOLID y patrones GRASP
9. Diagrama de clases final y cambios
10. Actualización del diseño durante Etapa 2

Anexo: cambios de implementación, correspondencia UML-código y síntesis de pruebas.

# 1. Análisis del enunciado

# Análisis del Enunciado – Agencia de Turismo Rumbo Sur

## 1. Introducción

Rumbo Sur es una agencia de turismo receptivo dedicada a organizar tours y excursiones de aventura, culturales y de naturaleza.

Actualmente, la agencia administra las reservas, los cupos y los pagos mediante planillas, lo que dificulta la organización de la información y el seguimiento de las operaciones.

Se propone desarrollar un sistema de gestión en Java, aplicando los conceptos de Programación Orientada a Objetos.

## 2. Problemática identificada

La gestión mediante planillas presenta dificultades para centralizar la información y controlar los distintos procesos de la agencia.

Entre las principales necesidades se encuentran:

- Organizar el catálogo de tours y excursiones.
- Controlar los cupos disponibles de cada salida.
- Registrar correctamente clientes y pasajeros.
- Calcular tarifas, promociones y descuentos.
- Llevar un seguimiento de los pagos y saldos.
- Administrar cancelaciones y devoluciones.
- Coordinar la asignación de guías y vehículos.
- Generar reportes para la administración.

## 3. Objetivo general

Desarrollar un sistema de gestión que permita administrar las operaciones de Rumbo Sur, centralizando la información y automatizando los procesos principales de la agencia.

## 4. Actores del sistema

Se identifican los actores que interactúan directamente con el sistema y los participantes externos involucrados en los procesos de la agencia.

### *Administrador*

Gestiona la información general del sistema, administra los parámetros de operación y consulta los reportes.

### *Vendedor*

Registra clientes, gestiona reservas, consulta disponibilidad y administra los pagos correspondientes.

### *Coordinador de operaciones*

Organiza las salidas programadas, asigna guías y vehículos, y consulta las listas de pasajeros.

### *Participantes externos*

El cliente, el guía de turismo y el proveedor intervienen en los procesos comerciales de la agencia, pero no se consideran actores directos del software.

## 5. Entidades principales

Las principales entidades identificadas para el modelado son:

- Tour

- Actividad del itinerario
- Salida
- Cliente
- Pasajero
- Reserva
- Tarifa
- Promoción
- Pago
- Servicio adicional
- Guía
- Vendedor
- Coordinador
- Administrador
- Vehículo
- Proveedor
- Cancelación

Estas entidades constituyen una identificación inicial y deberán revisarse durante la elaboración del diagrama de clases.

## 6. Procesos principales

### *Gestión de tours*

Permite administrar los tours, sus categorías, itinerarios, duración y tarifas.

### *Gestión de salidas*

Permite programar salidas, establecer cupos y asignar guías y vehículos.

### *Gestión de reservas*

Permite registrar clientes, pasajeros y reservas, verificando la disponibilidad de lugares.

### *Gestión de pagos*

Permite registrar señas, pagos posteriores y saldos pendientes.

### *Gestión de cancelaciones*

Permite cancelar o modificar reservas aplicando las políticas establecidas.

### *Gestión de reportes*

Permite consultar información sobre ocupación, ventas, ingresos y cancelaciones.

## 7. Alcance inicial del sistema

El sistema deberá contemplar las funcionalidades mínimas definidas en el enunciado:

- Administración de tours y salidas.
- Registro de clientes y reservas.
- Cálculo de precios y promociones.
- Registro de pagos.
- Contratación de servicios adicionales.
- Cancelación y modificación de reservas.
- Asignación de guías y transportes.

- Emisión de listas de pasajeros.
- Liquidación de honorarios.
- Generación de reportes administrativos.

## 8. Conclusión

El análisis inicial permite identificar las principales necesidades de Rumbo Sur y establecer una base para el modelado del sistema.

A partir de este relevamiento se podrán definir los requerimientos funcionales, las reglas de negocio y los diagramas UML correspondientes a la primera etapa del trabajo práctico.

# 2. Requerimientos funcionales

# Requerimientos Funcionales – Sistema de Gestión Rumbo Sur

## Introducción

Los requerimientos funcionales describen las operaciones que deberá realizar el sistema para satisfacer las necesidades comerciales, administrativas y operativas de la agencia de turismo Rumbo Sur.

## 1. Gestión de tours

### *RF01 – Administrar tours*

El sistema deberá permitir registrar, consultar y modificar tours, incluyendo código, nombre, descripción, duración, categoría y cupos mínimo y máximo.

### *RF02 – Gestionar itinerarios*

El sistema deberá permitir registrar las actividades de cada tour, indicando día, orden, horario, descripción y lugar, así como su vinculación con servicios de proveedores externos cuando corresponda.

### *RF03 – Gestionar tarifas*

El sistema deberá permitir establecer y consultar las tarifas base de los tours según temporada baja, media o alta, incluyendo sus períodos de vigencia.

### *RF04 – Programar salidas*

El sistema deberá permitir crear salidas asociadas a un tour, indicando fechas de inicio y finalización, cupo y estado. El guía y el vehículo podrán asignarse posteriormente, pero deberán estar asignados y validados antes de iniciar la salida.

### *RF05 – Gestionar estados de salidas*

El sistema deberá controlar los estados de las salidas y actualizar su disponibilidad según la cantidad de pasajeros registrados y las condiciones establecidas.

## 2. Gestión de clientes y reservas

### *RF06 – Registrar clientes*

El sistema deberá permitir registrar y consultar clientes, almacenando nombre, apellido, documento, teléfono, correo electrónico y país de residencia.

### *RF07 – Registrar pasajeros*

El sistema deberá permitir incorporar pasajeros a una reserva, registrando sus datos personales, fecha de nacimiento, tipo de pasajero, contacto de emergencia y restricciones relevantes.

### *RF08 – Crear reservas*

El sistema deberá permitir generar reservas asociadas a un cliente, una salida y un vendedor, incluyendo uno o más pasajeros y verificando la disponibilidad de cupos.

### *RF09 – Calcular precios*

El sistema deberá calcular el precio total de una reserva según la tarifa de temporada, el tipo de pasajero, las promociones aplicables y los servicios adicionales.

### *RF10 – Aplicar promociones*

El sistema deberá evaluar las promociones por reserva anticipada y por grupo, seleccionando la más beneficiosa cuando ambas resulten aplicables.

### *RF11 – Gestionar servicios adicionales*

El sistema deberá permitir contratar excursiones opcionales, traslados y seguros para cada pasajero, incluyendo los servicios obligatorios según el tipo de tour.

## 3. Gestión de pagos y cancelaciones

### *RF12 – Registrar pagos*

El sistema deberá permitir registrar señas y pagos posteriores, identificando fecha, importe y medio de pago.

### *RF13 – Controlar saldos*

El sistema deberá calcular el saldo pendiente de cada reserva y verificar el cumplimiento del plazo establecido para completar el pago.

### *RF14 – Confirmar o anular reservas*

El sistema deberá confirmar reservas al cumplirse la condición de la seña y anular aquellas que no completen el saldo dentro del plazo correspondiente, liberando sus cupos.

### *RF15 – Cancelar reservas*

El sistema deberá permitir registrar cancelaciones, sus motivos y fechas, y calcular las devoluciones según la política establecida.

### *RF16 – Modificar reservas*

El sistema deberá permitir gestionar modificaciones y reemplazos de pasajeros, verificando los plazos y requisitos del tour.

### *RF17 – Registrar ajustes*

El sistema deberá permitir corregir movimientos económicos de reservas pagadas mediante notas de ajuste, sin eliminar los registros originales.

## 4. Gestión operativa

### *RF18 – Administrar guías y vehículos*

El sistema deberá permitir registrar y consultar guías, sus habilitaciones, idiomas y honorarios, así como vehículos, sus tipos, patentes y capacidades.

### *RF19 – Asignar recursos*

El sistema deberá permitir asignar guías y vehículos a las salidas, verificando que cumplan las condiciones de disponibilidad, habilitación y capacidad.

### *RF20 – Emitir listas de pasajeros*

El sistema deberá generar la lista de pasajeros de cada salida, incluyendo documentos, contactos de emergencia e información relevante para el guía.

### *RF21 – Gestionar salidas*

El sistema deberá permitir iniciar y finalizar salidas, verificando las condiciones operativas obligatorias.

### *RF22 – Liquidar honorarios*

El sistema deberá calcular los honorarios de los guías correspondientes a salidas finalizadas, considerando días trabajados y ventas de servicios adicionales.

### *RF23 – Administrar proveedores*

El sistema deberá permitir registrar y consultar proveedores, incluyendo nombre, CUIT, contacto y tipo de servicio.

## 5. Reportes administrativos

### *RF24 – Generar reportes*

El sistema deberá permitir emitir reportes de ocupación por salida, ventas por vendedor, ingresos por tour y cancelaciones.

# 3. Reglas de negocio

# Reglas de Negocio – Sistema de Gestión Rumbo Sur

## Introducción

Las reglas de negocio establecen las condiciones, restricciones y políticas que deben cumplirse durante el funcionamiento del sistema de gestión de la agencia Rumbo Sur.

Su finalidad es garantizar la correcta administración de los tours, las reservas, los pagos y las operaciones turísticas.

## 1. Tours y salidas

### *RN01 – Organización de itinerarios*

Cada tour deberá contar con un itinerario compuesto por actividades ordenadas por día y podrá tener una duración de uno o varios días.

### *RN02 – Categorías de tours*

Los tours se clasificarán en aventura, cultural y naturaleza. Los tours de aventura deberán establecer una edad mínima y un nivel de dificultad.

### *RN03 – Programación de salidas*

Cada salida corresponderá a un tour y tendrá fechas programadas y cupo máximo. El guía y el vehículo deberán estar asignados antes de iniciar la salida, de acuerdo con RN30.

### *RN04 – Estados de las salidas*

Los estados posibles serán: programada, confirmada, completa, en curso, finalizada y cancelada.

### *RN05 – Confirmación de salidas*

Una salida se confirmará cuando alcance el cupo mínimo establecido.

### *RN06 – Cancelación por cupo insuficiente*

Si siete días antes de la fecha de inicio no se alcanza el cupo mínimo, la salida se cancelará y se devolverá el 100 % de los importes abonados.

### *RN07 – Control de cupos*

Cuando la totalidad de los cupos de una salida se encuentre ocupada por reservas confirmadas o retenida temporalmente por reservas pendientes vigentes, la salida pasará al estado completa y no admitirá nuevas reservas que excedan su capacidad máxima.

Las reservas pendientes retendrán cupos únicamente durante el plazo establecido en S04 y S09. Cuando una reserva sea anulada o venza su plazo de retención, los cupos correspondientes volverán a estar disponibles.

### *RN08 – Disponibilidad de guías*

Un guía no podrá tener asignadas dos salidas superpuestas y solo podrá dirigir tours de categorías para las cuales se encuentre habilitado.

### *RN09 – Capacidad del vehículo*

La capacidad del vehículo asignado deberá ser igual o superior al cupo máximo de la salida.

## 2. Reservas y precios

### *RN10 – Titularidad de reservas*

Cada reserva pertenecerá a un único cliente titular, estará asociada a una salida e incluirá uno o más pasajeros.

### *RN11 – Ocupación de cupos*

Cada reserva ocupará tantos lugares como pasajeros registrados tenga, incluidos aquellos que no abonen tarifa.

### *RN12 – Tarifas por temporada*

La tarifa base de cada tour dependerá de la temporada: baja, media o alta.

### *RN13 – Tarifas por tipo de pasajero*

Los adultos abonarán el 100 % de la tarifa base; los menores de 3 años no abonarán tarifa, aunque deberán registrarse; y los menores desde los 3 años hasta antes de cumplir 12 abonarán el 60 %. La edad se determinará al inicio de la salida (S03).

### *RN14 – Descuento para jubilados*

Los pasajeros jubilados recibirán un descuento del 10 %.

### *RN15 – Promoción por reserva anticipada*

Las reservas realizadas con más de 60 días de anticipación recibirán un descuento del 15 %.

### *RN16 – Promoción por grupo*

Las reservas de seis o más pasajeros recibirán un descuento del 10 %.

### *RN17 – Promociones no acumulables*

Las promociones por reserva anticipada y por grupo no podrán acumularse. Cuando ambas sean aplicables, se utilizará la más favorable para el cliente.

### *RN18 – Servicios adicionales*

Los servicios adicionales contratados se sumarán al precio correspondiente a cada pasajero.

### *RN19 – Seguro obligatorio*

Todos los pasajeros de tours de aventura deberán contar con seguro de viaje.

### *RN20 – Confirmación de reservas*

Una reserva se confirmará al abonar una seña equivalente al 30 % del importe total.

### *RN21 – Vencimiento del saldo*

El saldo pendiente deberá abonarse hasta diez días antes de la salida. En caso contrario, la reserva se anulará y los lugares ocupados volverán a estar disponibles.

### *RN22 – Medios de pago y recargos*

Los pagos podrán realizarse en cuotas mediante efectivo, transferencia o tarjeta. Los pagos con tarjeta en cuotas tendrán un recargo del 7 % sobre el capital de ese pago. El recargo se registra separado del importe abonado, no modifica el total de la reserva ni la base de cálculo de una devolución (S11).

## 3. Cancelaciones y modificaciones

### *RN23 – Cancelación anticipada*

Las cancelaciones realizadas hasta 30 días antes de la salida tendrán una devolución del 90 % de lo abonado.

### *RN24 – Cancelación entre 29 y 15 días*

Las cancelaciones realizadas dentro de este período tendrán una devolución del 50 %.

### *RN25 – Cancelación entre 14 y 7 días*

Las cancelaciones realizadas dentro de este período tendrán una devolución del 20 %.

### *RN26 – Cancelación con menos de 7 días*

Las cancelaciones realizadas con menos de siete días de anticipación no tendrán devolución.

### *RN27 – Cancelación por parte de la agencia*

Cuando la cancelación sea provocada por la agencia, la devolución será del 100 % de los importes abonados.

### *RN28 – Reemplazo de pasajeros*

Se permitirá reemplazar un pasajero por otro hasta tres días antes de la salida, sin cargo, siempre que el reemplazante cumpla las condiciones del tour.

## 4. Operación y liquidaciones

### *RN29 – Emisión de listas de pasajeros*

Antes de cada salida, el coordinador deberá emitir una lista con los documentos y contactos de emergencia de los pasajeros.

### *RN30 – Condiciones para iniciar una salida*

No se podrá iniciar una salida sin una lista de pasajeros emitida, un guía asignado y habilitado para la categoría del tour, y un vehículo asignado cuya capacidad cumpla con lo establecido en RN09.

### *RN31 – Liquidación de honorarios*

Los honorarios de los guías se liquidarán únicamente por salidas finalizadas.

### *RN32 – Cálculo de honorarios*

Los honorarios se calcularán mediante un monto fijo por día trabajado, más un 5 % de las ventas de servicios adicionales de la salida.

### *RN33 – Conservación de reservas pagadas*

Una reserva pagada no podrá eliminarse. Las correcciones deberán registrarse mediante notas de ajuste, que no eliminan el movimiento original (S22).

# 4. Supuestos adoptados

# Supuestos Adoptados – Sistema de Gestión Rumbo Sur

## 1. Introducción

A partir del análisis del enunciado de la agencia Rumbo Sur, se identificaron aspectos que requieren definiciones adicionales para garantizar un funcionamiento coherente del sistema.

Los siguientes supuestos son decisiones adoptadas por el grupo antes de la implementación.

## 2. Supuestos adoptados

### *S01 – Acceso al sistema*

Se considera que el administrador, los vendedores y el coordinador de operaciones serán usuarios directos del sistema. Los guías son personal de la agencia pero no usan el sistema: sus datos los carga el coordinador.

### *S02 – Identificación de registros*

Cada cliente, tour, salida, reserva y pago tendrá un identificador único para facilitar su administración y evitar duplicaciones.

### *S03 – Cálculo de edades*

La edad de cada pasajero se determinará según la fecha de inicio de la salida, tanto para calcular tarifas como para verificar restricciones de participación.

### *S04 – Reservas pendientes*

Las reservas pendientes de seña conservarán temporalmente los cupos solicitados hasta su fecha de vencimiento, que es la menor entre 2 días calendario desde su creación y 10 días antes del inicio de la salida (fecha límite de pago del saldo, RN21). La reserva se considera vencida cuando la fecha actual supera su fecha de vencimiento.

Ejemplo: una reserva creada el 2 de marzo para una salida el 10 de abril vence el 4 de marzo (el menor entre el 4 de marzo y el 31 de marzo). El 5 de marzo ya está vencida.

Si la reserva se realiza dentro de los diez días anteriores a la salida, se deberá abonar el importe total al momento de registrarla, debido a que el plazo habitual para cancelar el saldo ya habrá vencido.

Cuando una reserva pendiente no cumpla con las condiciones de pago dentro del plazo establecido, será anulada y los cupos retenidos volverán a estar disponibles.

### *S05 – Aplicación de descuentos*

Los descuentos por tipo de pasajero se calcularán antes de evaluar las promociones comerciales. Entre las promociones por anticipación y por grupo se aplicará únicamente la más beneficiosa.

### *S06 – Servicios adicionales*

Las promociones comerciales se aplicarán sobre el valor del tour, mientras que los servicios adicionales se incorporarán posteriormente al importe de la reserva.

### *S07 – Persistencia de información*

Toda la información del sistema (tours, salidas, clientes, reservas, pagos, guías, vehículos y proveedores) se guardará en un archivo mediante serialización de objetos Java, para conservarla entre ejecuciones. La lógica de negocio accede al almacenamiento solo a través de la interfaz `RepositorioAgencia`.

### *S08 – Interfaz de usuario*

La interfaz del sistema será por consola con menú de opciones, organizada según las funcionalidades y responsabilidades de los usuarios. Esta modalidad fue confirmada por la profesora.

### *S09 – Vencimiento por falta de pago*

El sistema verificará el vencimiento de las reservas pendientes al iniciar operaciones relacionadas con reservas y al consultar la disponibilidad de cupos. También verificará las reservas confirmadas cuyo saldo todavía esté pendiente al alcanzar el límite de pago.

Si una reserva pendiente no alcanzó la seña y la fecha actual supera su vencimiento de retención (S04), será anulada automáticamente. Si una reserva confirmada conserva saldo impago y la fecha actual supera el límite de diez días antes de la salida (RN21), también será anulada. En ambos casos la verificación libera los cupos retenidos.

La anulación liberará los cupos retenidos y dejará registrado el cambio de estado, permitiendo mantener actualizada la disponibilidad de cada salida.

### *S10 – Interpretación de fechas de cancelación*

Los días de anticipación para cancelaciones se calcularán como días calendario completos entre la fecha de cancelación y la fecha de inicio de la salida.

Este criterio será utilizado de manera uniforme para RN23 a RN28 y fue adoptado por el grupo.

Los tramos se interpretan de la siguiente manera: con 30 días o más de anticipación se devuelve el 90 %; de 29 a 15 días, el 50 %; de 14 a 7 días, el 20 %; con menos de 7 días no hay devolución. Si la cancelación la provoca la agencia, la devolución es del 100 % sin importar los días.

### *S11 – Reglas de precio y recargo*

El descuento de jubilados se considera una tarifa especial por tipo de pasajero; las promociones comerciales RN15 y RN16 no se acumulan entre sí.

Conforme a S05 y S06, primero se determina la tarifa por pasajero, luego se aplica la promoción comercial elegida sobre el tour y finalmente se suman los adicionales.

El recargo del 7 % se aplicará únicamente al importe de capital que se registre en un pago con tarjeta en cuotas. El total de la reserva representa el precio de los servicios, sin recargo; cada `Pago` guarda por separado su `importe` (capital abonado, sin recargo) y su `recargo`. El saldo de la reserva se calculará sobre el capital pendiente, mientras que el importe efectivamente cobrado será capital más recargo.

Este orden de cálculo es una interpretación adoptada por el grupo. La devolución se calcula sobre el capital abonado, sin recargos, de acuerdo con S17.

### *S12 – Estados de la reserva*

Los estados de la reserva son los del enunciado. Pendiente: creada, sin la seña y con retención temporal de cupos. Confirmada: al registrarse la seña del 30 %, aunque todavía tenga saldo pendiente. Anulada: por falta de seña o por saldo impago fuera de plazo (S09), por cancelación solicitada por el cliente o por la agencia, o por cancelación de la salida. Finalizada: cuando finaliza la salida.

No existe un estado «cancelada» para la reserva: toda cancelación deja la reserva en estado anulada y registra una cancelación con fecha, motivo, porcentaje e importe devuelto.

### *S13 – Medio de devolución*

La devolución se acredita por el mismo medio de pago utilizado en la reserva. Si se abonó con más de un medio, se utiliza el del último pago registrado.

### *S14 – Comisión del vendedor*

Cada vendedor tiene una comisión, registrada como dato de su ficha. El sistema no liquida comisiones: el reporte de ventas por vendedor informa los importes vendidos.

### *S15 – Reservas que pueden cancelarse*

Una reserva puede cancelarse si está en estado pendiente o confirmada y su salida no está en curso ni finalizada. Una reserva anulada o finalizada no puede cancelarse.

### *S16 – Tipos de pasajero y edades*

El tipo «menor» corresponde a pasajeros menores de 12 años (RN13). Los pasajeros de 12 años o más se registran como adulto o jubilado. Un pasajero menor de 3 años no abona tarifa, pero ocupa un lugar y debe registrarse. El sistema valida que el tipo informado sea coherente con la fecha de nacimiento.

### *S17 – Base de cálculo de la devolución*

La devolución se calcula sobre los importes abonados sin incluir el recargo por financiación con tarjeta. El porcentaje se aplica sobre ese total (RN23 a RN27).

### *S18 – Cupos de la salida y del tour*

El cupo máximo de una salida no puede superar el cupo máximo de su tour. El cupo mínimo para confirmarla es el del tour (RN05).

### *S19 – Cancelación de una salida por cupo mínimo*

Cuando el sistema detecta que faltan 7 días o menos para la salida sin alcanzar el cupo mínimo (RN06), cambia la salida a cancelada y cancela todas sus reservas vigentes como cancelación de la agencia, con devolución del 100 % (RN27).

### *S20 – Temporada de una salida*

La temporada y la tarifa base de una salida se determinan con la tarifa del tour que esté vigente en la fecha de inicio de la salida (RN12).

### *S21 – Servicios adicionales de un tour*

Cada servicio adicional pertenece a un tour y se marca como opcional u obligatorio para ese tour. Los servicios se contratan por pasajero. Los servicios obligatorios se incorporan automáticamente a todos los pasajeros al crear la reserva; en los tours de aventura, el seguro de viaje es un servicio obligatorio (RN19).

### *S22 – Notas de ajuste*

Una nota de ajuste tiene un importe que puede ser positivo (aumenta el total de la reserva) o negativo (lo reduce). No elimina ni modifica ningún pago ya registrado (RN32). El saldo de la reserva es: total + suma de los ajustes − capital abonado.

# 5. Casos de uso

# 2. Diagrama de casos de uso

Los casos de uso se identificaron a partir de los 24 requerimientos funcionales (RF01–RF24), las reglas de negocio RN01–RN33 y los supuestos S01–S22.

Se consideran actores directos del sistema al Administrador, al Vendedor y al Coordinador de operaciones.

El Cliente, el Guía de turismo y el Proveedor participan de los procesos del negocio, pero no interactúan directamente con el software en esta primera versión.

Los casos de uso se organizaron por módulos para facilitar la lectura de los diagramas. El Vendedor aparece en gestión comercial y en pagos, pero representa un único actor.

Los códigos CU identifican casos de uso y no deben confundirse con los códigos RF, RN o S.

## 2.1. Identificación de casos de uso

| Actor | Responsabilidad | Casos de uso |
|---|---|---|
| Administrador | Catálogo de tours, itinerarios, tarifas, proveedores y reportes | CU01, CU02, CU03, CU23, CU24 |
| Vendedor | Clientes, reservas, precios, pagos, cancelaciones y ajustes | CU06–CU17 |
| Coordinador de operaciones | Salidas, recursos, listas de pasajeros y honorarios | CU04, CU05, CU18–CU22 |

### *Gestión administrativa*

Comprende la administración del catálogo de tours, itinerarios, tarifas, proveedores y reportes.

### *Gestión comercial y reservas*

Comprende el registro de clientes y pasajeros, la creación y modificación de reservas, el cálculo de precios, la evaluación de promociones, los servicios adicionales.

### *Gestión de pagos y cancelaciones*

Comprende el registro de pagos, el control de saldos, la confirmación o anulación de reservas, las cancelaciones y los ajustes económicos.

### *Gestión operativa*

Comprende la programación de salidas, la administración y asignación de recursos, la emisión de listas de pasajeros, el inicio y finalización de salidas y la liquidación de honorarios.

## 2.2. Diagramas UML de casos de uso

Los diagramas UML representan gráficamente las interacciones entre los actores y las funcionalidades del Sistema de Gestión Rumbo Sur.

Se utilizaron asociaciones mediante líneas continuas para indicar qué funcionalidades puede ejecutar cada actor.

Las relaciones «include» se utilizan cuando un caso de uso requiere obligatoriamente otro comportamiento modelado como caso de uso independiente.

Las relaciones «extend» se utilizan para comportamientos condicionales: el caso que extiende solo se ejecuta si se da una condición (por ejemplo, los servicios adicionales solo si el vendedor elige alguno). La flecha va desde el caso que extiende hacia el caso base.

**Relaciones modeladas:**

- **«include»:** CU08 (Crear reserva) incluye CU07 (Registrar pasajeros) y CU09 (Calcular precio de reserva). CU09 incluye CU10 (Evaluar promociones). CU12 (Registrar pagos) incluye CU13 (Controlar saldos), considerando la actualización del saldo como parte del registro de cada pago.

- **«extend»:** CU11 (Gestionar servicios adicionales) extiende CU08 solo cuando el vendedor elige servicios opcionales. Los servicios obligatorios (por ejemplo, el seguro en los tours de aventura) no son una extensión: se incorporan automáticamente al crear la reserva como parte de CU08 (S21).

- **CU14 (Confirmar o anular reservas):** se mantiene como caso de uso independiente. La confirmación se realiza cuando se cumplen las condiciones de pago y demás requisitos establecidos. La anulación se produce cuando corresponde según las reglas de negocio, por ejemplo, ante el vencimiento del plazo de pago. Estas situaciones se documentan en sus flujos principales y alternativos.

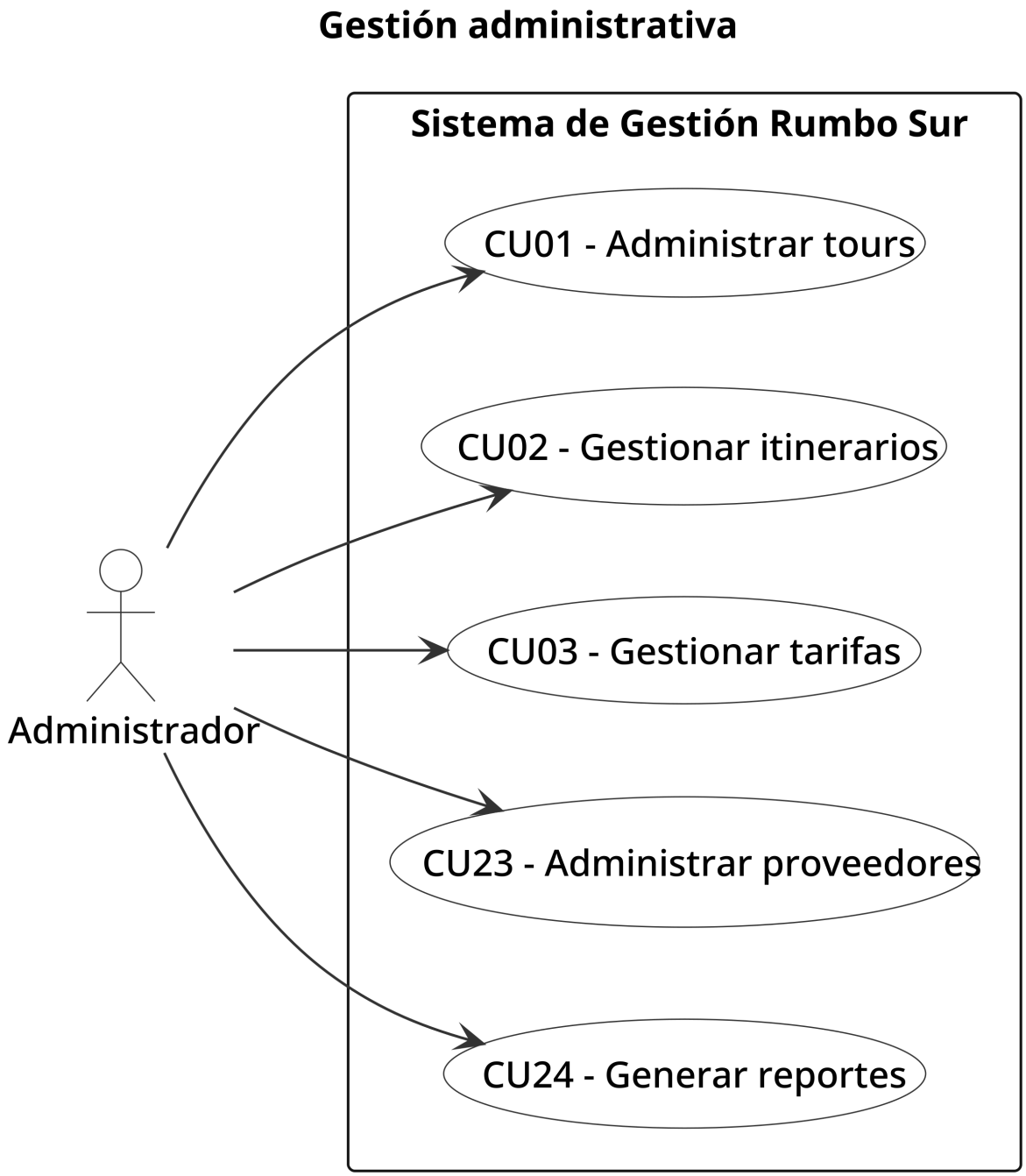
- **CU19 (Asignar recursos):** se mantiene como caso de uso independiente del CU04 (Programar salidas), ya que la asignación de guías y vehículos puede gestionarse después de programar una salida.

Las relaciones «include» representan comportamientos obligatorios y las relaciones «extend» representan comportamientos condicionales. Los flujos alternativos y las excepciones se detallan en las fichas textuales correspondientes.

### *2.2.1. Gestión administrativa*

El Administrador gestiona tours, itinerarios, tarifas y proveedores, y consulta reportes administrativos.

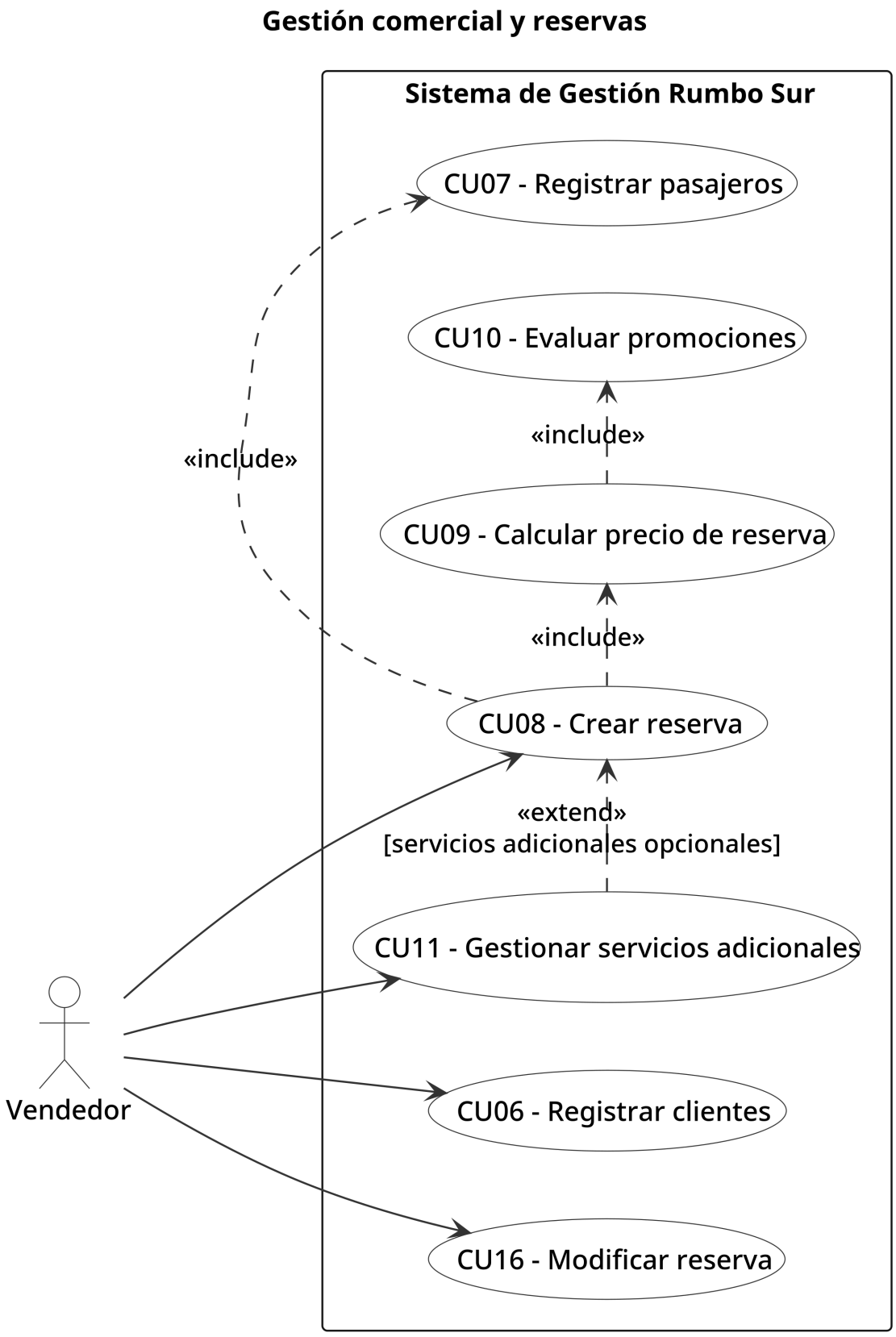
**Figura 1. Diagrama de casos de uso de gestión administrativa.**




### *2.2.2. Gestión comercial y reservas*

El Vendedor registra clientes y pasajeros, crea y modifica reservas, calcula precios, evalúa promociones, y gestiona servicios adicionales.

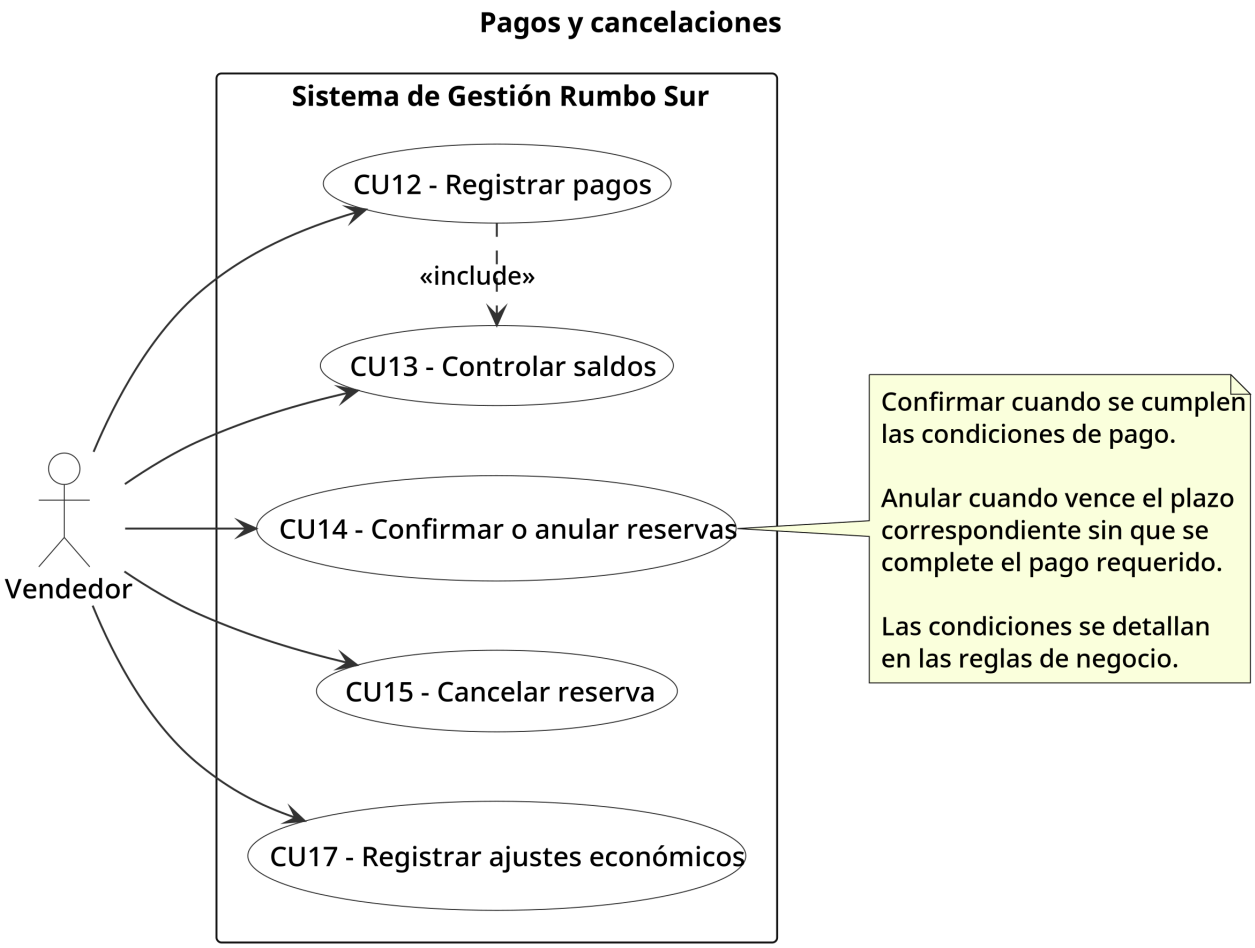
**Figura 2. Diagrama de casos de uso de gestión comercial y reservas.**




### *2.2.3. Pagos y cancelaciones*

El Vendedor registra pagos, controla saldos, confirma o anula reservas, gestiona cancelaciones y registra ajustes económicos.

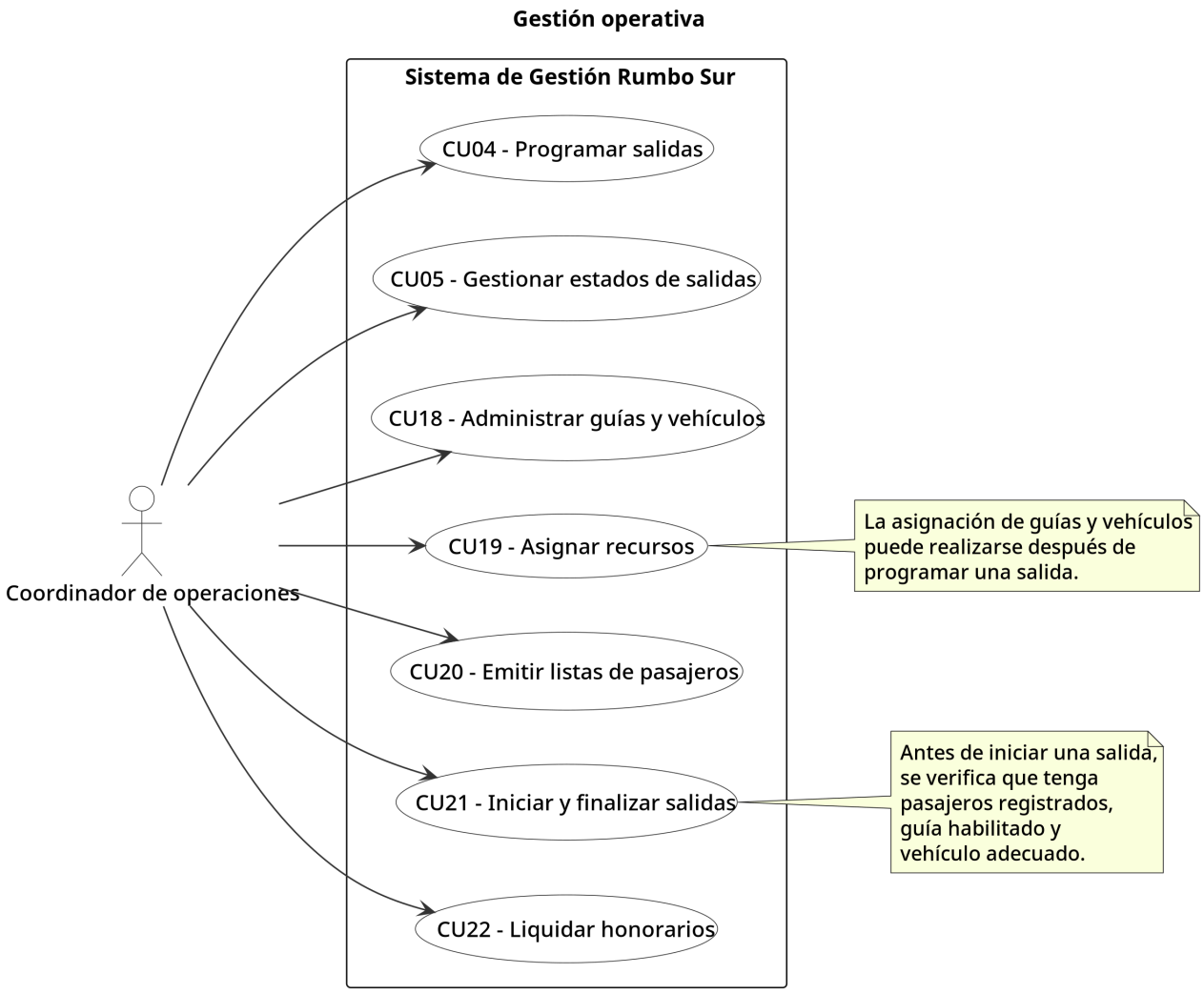
**Figura 3. Diagrama de casos de uso de pagos y cancelaciones.**




### *2.2.4. Gestión operativa*

El Coordinador de operaciones programa salidas, administra y asigna recursos, emite listas de pasajeros, inicia y finaliza salidas y liquida honorarios.

**Figura 4. Diagrama de casos de uso de gestión operativa.**




## 2.3. Descripciones textuales de casos de uso

Las siguientes fichas describen el comportamiento esperado del sistema. Cada una incluye actor principal, objetivo, precondición, flujo principal, alternativas o excepciones, postcondición y trazabilidad.

### *CU01 – Administrar tours*

**Actor principal:** Administrador.

**Objetivo:** Registrar, consultar y modificar los tours ofrecidos por la agencia.

**Precondición:** Administrador autenticado.

**Flujo principal:**

1. El administrador selecciona la administración de tours.
2. Selecciona alta, consulta o modificación.
3. Informa código, nombre, descripción, categoría, duración y cupos.
4. El sistema valida los datos.
5. El sistema registra o actualiza el tour.

**Flujos alternativos:**

• Si el código está duplicado, se informa el error.

• Si existen datos inválidos, se solicita corregirlos.

• Si el tour está referenciado por otras operaciones, se restringen los cambios incompatibles.

**Postcondición:** Tour registrado o actualizado.

**Trazabilidad:** RF01 · RN01–RN02 · S02.

### *CU02 – Gestionar itinerarios*

**Actor principal:** Administrador.

**Objetivo:** Registrar y mantener las actividades que componen los itinerarios.

**Precondición:** Tour existente.

**Flujo principal:**

1. El administrador selecciona un tour.
2. Agrega, modifica u ordena actividades.
3. Informa día, horario, lugar y descripción.
4. Vincula servicios de proveedores cuando corresponda.
5. El sistema valida y guarda el itinerario.

**Flujos alternativos:**

- Si una actividad está incompleta, se solicita corrección.
- Si el día informado excede la duración del tour, se rechaza el registro.

**Postcondición:** Itinerario actualizado.

**Trazabilidad:** RF02 · RN01 · S02.

### *CU03 – Gestionar tarifas*

**Actor principal:** Administrador.

**Objetivo:** Administrar las tarifas de los tours según temporada.

**Precondición:** Tour existente.

**Flujo principal:**

1. El administrador selecciona un tour.
2. Selecciona temporada baja, media o alta.
3. Informa período de vigencia y tarifa base.
4. El sistema valida los datos.
5. El sistema registra la tarifa.

**Flujos alternativos:**

- Si existen períodos incompatibles o superpuestos, se solicita corrección.
- Si el importe no es válido, se rechaza el registro.

**Postcondición:** Tarifa disponible para el cálculo de reservas.

**Trazabilidad:** RF03 · RN12 · S11, S20.

### *CU04 – Programar salidas*

**Actor principal:** Coordinador de operaciones.

**Objetivo:** Crear salidas programadas para los tours.

**Precondición:** Tour registrado y fechas válidas.

**Flujo principal:**

1. El coordinador selecciona un tour.
2. Informa las fechas de inicio y finalización.
3. Establece los cupos correspondientes.

4. El sistema valida fechas y capacidad.
5. Registra la salida en estado programada.
6. Cuando corresponde, se asignan recursos mediante CU19.

**Flujos alternativos:**

• Si los cupos no respetan los límites del tour, se rechaza el registro.

• Si las fechas son inconsistentes, se solicita corrección.

**Postcondición:** Salida programada.

**Trazabilidad:** RF04 · RN03–RN04 · S02.

### *CU05 – Gestionar estados de salidas*

**Actor principal:** Coordinador de operaciones.

**Objetivo:** Controlar los estados y la disponibilidad de las salidas.

**Precondición:** Salida registrada.

**Flujo principal:**

1. El coordinador consulta una salida.
2. El sistema verifica su ocupación y fecha.
3. Evalúa el cumplimiento del cupo mínimo y máximo.
4. Actualiza el estado cuando corresponda.
5. Muestra la disponibilidad resultante.

**Flujos alternativos:**

• Si faltan siete días y no se alcanzó el cupo mínimo, se cancela la salida y corresponde devolver el 100 % de los pagos.

• Si se alcanzó el cupo máximo, la salida queda completa y no admite nuevas reservas.

**Postcondición:** Estado y disponibilidad actualizados.

**Trazabilidad:** RF05 · RN04–RN07 · S04, S09, S18, S19.

### *CU06 – Registrar clientes*

**Actor principal:** Vendedor.

**Objetivo:** Registrar y consultar los datos de los clientes.

**Precondición:** Vendedor autenticado.

**Flujo principal:**

1. El vendedor selecciona la gestión de clientes.
2. Ingresa nombre, apellido, documento, teléfono, correo y país de residencia.
3. El sistema valida los datos.
4. Registra el cliente.
5. Permite consultarlo para futuras reservas.

**Flujos alternativos:**

• Si el documento ya existe, el sistema informa el registro correspondiente.

• Si faltan datos obligatorios, solicita completarlos.

**Postcondición:** Cliente disponible para realizar reservas.

**Trazabilidad:** RF06 · RN10 · S02.

### *CU07 – Registrar pasajeros*

**Actor principal:** Vendedor.

**Objetivo:** Incorporar pasajeros a una reserva.

**Precondición:** Reserva en preparación o susceptible de modificación.

**Flujo principal:**

1. El vendedor selecciona la reserva.
2. Registra los datos personales del pasajero.
3. Informa fecha de nacimiento, tipo de pasajero, contacto de emergencia y restricciones.
4. El sistema verifica los datos y las condiciones del tour.
5. Asocia el pasajero a la reserva.

**Flujos alternativos:**

- Si faltan datos obligatorios, se solicita completarlos.
- Si el pasajero no cumple los requisitos del tour, se impide su incorporación.

**Postcondición:** Pasajeros asociados a la reserva.

**Trazabilidad:** RF07 · RN10–RN13, RN19 · S03.

### *CU08 – Crear reserva*

**Actor principal:** Vendedor.

**Objetivo:** Registrar una reserva asociada a un cliente, una salida y sus pasajeros, verificando la disponibilidad, las restricciones de participación y las condiciones de pago.

**Precondición:** Cliente y salida existentes; salida habilitada para reservas.

**Flujo principal:**

1. El vendedor selecciona un cliente registrado.
2. Selecciona una salida habilitada para reservas.
3. Registra los pasajeros mediante CU07.
4. El sistema verifica la disponibilidad de cupos y los requisitos de participación de cada pasajero.
5. El sistema incorpora automáticamente los servicios obligatorios del tour (por ejemplo, el seguro en los tours de aventura) y el vendedor puede agregar servicios opcionales mediante CU11.
6. El sistema calcula el precio total mediante CU09, considerando las tarifas, las promociones aplicables y los servicios adicionales.
7. El sistema muestra el importe desglosado y las condiciones de pago.
8. El vendedor confirma el registro.
9. Si faltan más de diez días para la salida, el sistema genera una reserva pendiente y retiene temporalmente los cupos según S04, siempre que se cumplan las condiciones de vigencia y pago.
10. Cuando faltan diez días o menos para la salida, se exige el pago total durante la operación y la reserva solamente se confirma cuando se registra dicho pago.

**Flujos alternativos:**

- **Cupos insuficientes:** Se informa la falta de disponibilidad y no se crea la reserva.
- **Restricciones de participación:** Se impide registrar la reserva con pasajeros que no cumplen las condiciones.
- **Pago total no registrado cuando es obligatorio:** No se registra la reserva y se informa al vendedor que debe completarse el pago.
- **Datos incompletos:** Se solicita completar la información antes de guardar.

**Postcondición:** Reserva pendiente con cupos retenidos temporalmente o reserva confirmada cuando corresponda el pago total.

**Trazabilidad:** RF08 · RN10–RN22 · S03–S06, S09, S11, S16, S21.

### *CU09 – Calcular precio de reserva*

**Actor principal:** Vendedor.

**Objetivo:** Determinar el importe total de una reserva.

**Precondición:** Salida, pasajeros y tarifa disponibles.

**Flujo principal:**

1. El sistema identifica la temporada aplicable.
2. Determina la edad y el tipo de cada pasajero.
3. Aplica las tarifas correspondientes.
4. Evalúa las promociones mediante CU10.
5. Incorpora los servicios adicionales.
6. Muestra el importe total desglosado.

**Flujos alternativos:**

- Si no existe tarifa vigente, se impide finalizar el cálculo.
- Si faltan datos necesarios, se solicita completarlos.

**Postcondición:** Importe calculado y desglosado.

**Trazabilidad:** RF09 · RN12–RN19 · S03, S05, S06, S11.

### *CU10 – Evaluar promociones*

**Actor principal:** Vendedor.

**Objetivo:** Determinar qué promoción comercial corresponde aplicar.

**Precondición:** Fecha de reserva, pasajeros e importe base conocidos.

**Flujo principal:**

1. El sistema verifica si la reserva se realiza con más de 60 días de anticipación.
2. Verifica si incluye seis o más pasajeros.
3. Identifica las promociones aplicables.
4. Selecciona únicamente la más beneficiosa.
5. Devuelve el descuento correspondiente al cálculo del precio.

**Flujos alternativos:**

- Si ninguna promoción es aplicable, se mantiene el importe sin promoción comercial.

**Postcondición:** Promoción seleccionada o ausencia de promoción.

**Trazabilidad:** RF10 · RN15–RN17 · S05, S11.

### *CU11 – Gestionar servicios adicionales*

**Actor principal:** Vendedor.

**Objetivo:** Incorporar servicios adicionales a los pasajeros.

**Precondición:** Tour y pasajeros identificados.

**Flujo principal:**

1. El vendedor consulta los servicios disponibles.
2. Selecciona excursiones opcionales, traslados o seguros.

3. Asocia los servicios a los pasajeros correspondientes.
4. El sistema verifica los servicios obligatorios.
5. Actualiza el importe de la reserva.

**Flujos alternativos:**

• Si el tour es de aventura y falta el seguro obligatorio, se exige incorporarlo.

• Si el servicio seleccionado no está disponible, se rechaza la selección.

**Postcondición:** Servicios adicionales asociados a los pasajeros.

**Trazabilidad:** RF11 · RN18–RN19 · S06, S21.

### *CU12 – Registrar pagos*

**Actor principal:** Vendedor.

**Objetivo:** Registrar señas y pagos de reservas.

**Precondición:** Reserva vigente e importe adeudado determinado.

**Flujo principal:**

1. El vendedor selecciona una reserva.
2. Ingresa fecha, importe y medio de pago.
3. Si corresponde, indica la cantidad de cuotas.
4. El sistema calcula el recargo por financiación con tarjeta.
5. Registra el pago.
6. Actualiza el saldo mediante CU13.
7. Evalúa las condiciones de confirmación de la reserva.

**Flujos alternativos:**

• Si el pago es inválido, se solicita corrección.

• Si se alcanza la seña requerida, se evalúa la confirmación mediante CU14.

**Postcondición:** Pago registrado y saldo actualizado.

**Criterio económico:** El importe ingresado es el capital abonado. Si el pago es con tarjeta en cuotas, el sistema calcula el recargo del 7 % sobre ese capital y registra ambos por separado (el monto cobrado es capital + recargo). La seña y el saldo se calculan siempre sobre el capital.

**Trazabilidad:** RF12 · RN20–RN22 · S04, S11.

### *CU13 – Controlar saldos*

**Actor principal:** Vendedor.

**Objetivo:** Consultar y controlar los saldos pendientes de las reservas.

**Precondición:** Reserva con importe y pagos registrados.

**Flujo principal:**

1. El vendedor selecciona una reserva.
2. El sistema obtiene su importe total.
3. Consulta los pagos registrados.
4. Calcula el saldo pendiente.
5. Verifica la fecha límite de pago.
6. Informa el estado de cumplimiento.

**Flujos alternativos:**

- Si el saldo se encuentra vencido, se evalúa la anulación correspondiente mediante CU14.

**Postcondición:** Saldo y estado de cumplimiento disponibles.

**Trazabilidad:** RF13 · RN20–RN21 · S04, S09.

### *CU14 – Confirmar o anular reservas*

**Actor principal:** Vendedor.

**Objetivo:** Gestionar la confirmación o anulación de reservas según las condiciones de pago y los plazos establecidos.

**Precondición:** Reserva existente con importe, pagos y fechas disponibles.

**Flujo principal:**

1. El vendedor selecciona la reserva.
2. El sistema consulta los pagos registrados.
3. Verifica los plazos aplicables.
4. Si se alcanzó la seña mínima del 30 %, o el pago total cuando corresponda, confirma la reserva.
5. Actualiza el estado y conserva los cupos asignados.

**Flujos alternativos:**

- Si el pago es insuficiente y el plazo continúa vigente, la reserva permanece pendiente.
- Si vence el plazo de retención de una reserva pendiente sin seña, o el plazo para cancelar el saldo de una reserva confirmada, el sistema anula la reserva y libera los cupos.
- Si la reserva ya está anulada, se impide su confirmación.

**Postcondición:** Reserva confirmada, pendiente o anulada según las condiciones verificadas.

**Trazabilidad:** RF14 · RN20–RN21 · S04, S09.

### *CU15 – Cancelar reserva*

**Actor principal:** Vendedor.

**Objetivo:** Registrar cancelaciones y calcular devoluciones.

**Precondición:** Reserva existente susceptible de cancelación.

**Flujo principal:**

1. El vendedor selecciona la reserva.
2. Indica motivo y fecha de cancelación.
3. El sistema calcula los días calendario hasta la salida.
4. Determina el porcentaje de devolución correspondiente.
5. Registra la cancelación y la devolución.
6. Libera los cupos de la reserva.

**Flujos alternativos:**

- Si la agencia cancela, corresponde una devolución del 100 %.
- Si el cliente cancela con menos de siete días de anticipación, no corresponde devolución.

**Postcondición:** Reserva cancelada y devolución registrada según la política aplicable.

**Trazabilidad:** RF15 · RN23–RN27 · S10, S13, S15, S17, S19.

### *CU16 – Modificar reserva*

**Actor principal:** Vendedor.

**Objetivo:** Modificar reservas y gestionar reemplazos de pasajeros.

**Precondición:** Reserva existente y salida no iniciada.

**Flujo principal:**

1. El vendedor selecciona una reserva.
2. Indica la modificación solicitada.
3. El sistema verifica plazos y condiciones.
4. Valida los requisitos del pasajero y los cupos cuando corresponda.
5. Recalcula importes si es necesario.
6. Registra la modificación.

**Flujos alternativos:**

- Si se solicita un reemplazo fuera del plazo permitido, se rechaza.
- Si el nuevo pasajero no cumple las condiciones del tour, se impide el cambio.
- Si no hay cupos suficientes para la modificación, no se aplica.

**Postcondición:** Reserva actualizada y modificación registrada.

**Trazabilidad:** RF16 · RN10–RN11, RN28 · S03, S10, S16.

### *CU17 – Registrar ajustes económicos*

**Actor principal:** Vendedor.

**Objetivo:** Registrar correcciones económicas sin eliminar movimientos originales.

**Precondición:** Reserva pagada con un movimiento que requiere corrección.

**Flujo principal:**

1. El vendedor selecciona el movimiento.
2. Informa el motivo del ajuste.
3. Indica el importe correspondiente.
4. El sistema valida la operación.
5. Registra una nota de ajuste sin eliminar el pago original.

**Flujos alternativos:**

- Si falta el motivo o el ajuste es inconsistente, se rechaza.

**Postcondición:** Ajuste registrado y trazabilidad económica conservada.

**Trazabilidad:** RF17 · RN33 · S02.

### *CU18 – Administrar guías y vehículos*

**Actor principal:** Coordinador de operaciones.

**Objetivo:** Registrar y consultar los recursos operativos.

**Precondición:** Coordinador autenticado.

**Flujo principal:**

1. El coordinador accede a la gestión de recursos.
2. Registra o consulta guías, habilitaciones, idiomas y honorarios.
3. Registra o consulta vehículos, patentes, tipos y capacidades.
4. El sistema valida la información.
5. Guarda los datos correspondientes.

**Flujos alternativos:**

- Si existen datos duplicados o inválidos, se solicita corrección.
- Si faltan habilitaciones necesarias, se informa la situación.

**Postcondición:** Recursos registrados y disponibles para asignación.

**Trazabilidad:** RF18 · RN08–RN09 · S02.

### *CU19 – Asignar recursos*

**Actor principal:** Coordinador de operaciones.

**Objetivo:** Asignar guías y vehículos a las salidas.

**Precondición:** Salida, guía y vehículo registrados.

**Flujo principal:**

1. El coordinador selecciona una salida.
2. Selecciona el guía y el vehículo.
3. El sistema verifica la disponibilidad del guía.
4. Controla sus habilitaciones.
5. Verifica la capacidad del vehículo.
6. Registra la asignación.

**Flujos alternativos:**

• Si el guía tiene otra salida superpuesta, se rechaza.

• Si no está habilitado para el tour, se impide la asignación.

• Si el vehículo no tiene capacidad suficiente, se solicita otro recurso.

**Postcondición:** Guía y vehículo asignados a la salida.

**Trazabilidad:** RF19 · RN08–RN09 · S02.

### *CU20 – Emitir lista de pasajeros*

**Actor principal:** Coordinador de operaciones.

**Objetivo:** Generar la lista operativa de pasajeros de una salida.

**Precondición:** Salida con reservas vigentes.

**Flujo principal:**

1. El coordinador selecciona la salida.
2. El sistema obtiene los pasajeros correspondientes.
3. Reúne documentos, contactos de emergencia y restricciones relevantes.
4. Genera la lista.
5. Registra que la lista fue emitida.

**Flujos alternativos:**

• Si faltan datos obligatorios, se solicita completarlos antes de emitirla.

**Postcondición:** Lista de pasajeros disponible para la operación.

**Trazabilidad:** RF20 · RN29–RN30.

### *CU21 – Iniciar y finalizar salidas*

**Actor principal:** Coordinador de operaciones.

**Objetivo:** Registrar el inicio y la finalización de las salidas, verificando las condiciones operativas necesarias.

**Precondición:** Salida existente en un estado compatible con la operación.

**Flujo principal:**

1. El coordinador selecciona una salida.

2. Solicita iniciar la operación.
3. El sistema verifica que se haya emitido la lista de pasajeros correspondiente.
4. Verifica que exista un guía asignado y habilitado para la categoría del tour.
5. Verifica que exista un vehículo asignado cuya capacidad cumpla con RN09.
6. El sistema comprueba que el estado de la salida permita iniciar la operación.
7. Si todas las condiciones se cumplen, cambia el estado de la salida a **en curso**.
8. Al concluir la operación, el coordinador registra la finalización.
9. El sistema verifica que la salida se encuentre **en curso**, cambia su estado a **finalizada** y finaliza las reservas vigentes asociadas a la salida.

**Flujos alternativos:**

- **Lista de pasajeros no emitida:** El sistema impide iniciar la salida e informa la condición pendiente.
- **Guía no asignado o no habilitado:** El sistema impide iniciar la salida hasta que se asigne un guía habilitado para la categoría del tour.
- **Vehículo no asignado o con capacidad insuficiente:** El sistema impide iniciar la salida hasta que se asigne un vehículo que cumpla RN09.
- **Estado incompatible:** El sistema rechaza la transición e informa que la operación no está permitida.

**Postcondición:** Salida en curso o finalizada, según la operación registrada; al finalizar, sus reservas vigentes quedan en estado finalizada.

**Trazabilidad:** RF21 · RN04, RN08–RN09, RN29–RN31.

### *CU22 – Liquidar honorarios*

**Actor principal:** Coordinador de operaciones.

**Objetivo:** Calcular y registrar los honorarios de los guías.

**Precondición:** Salida finalizada con guía asignado.

**Flujo principal:**

1. El coordinador selecciona una salida finalizada.
2. El sistema obtiene los días trabajados.
3. Consulta el honorario fijo diario.
4. Calcula el 5 % de las ventas de servicios adicionales.
5. Determina el importe total.
6. Registra la liquidación.

**Flujos alternativos:**

- Si la salida no está finalizada, se impide liquidar.
- Si faltan datos necesarios, se solicita completarlos.

**Postcondición:** Honorarios calculados y registrados.

**Trazabilidad:** RF22 · RN31–RN32 · S02.

### *CU23 – Administrar proveedores*

**Actor principal:** Administrador.

**Objetivo:** Registrar y consultar proveedores de servicios turísticos.

**Precondición:** Administrador autenticado.

**Flujo principal:**

1. El administrador accede a la gestión de proveedores.

2. Registra o consulta nombre, CUIT, contacto y tipo de servicio.
3. El sistema valida la información.
4. Registra o actualiza el proveedor.

**Flujos alternativos:**

• Si el CUIT es inválido o está duplicado, se solicita corrección.

**Postcondición:** Proveedor disponible para su vinculación con los servicios turísticos.

**Trazabilidad:** RF23 · RN01 · S02.

### *CU24 – Generar reportes*

**Actor principal:** Administrador.

**Objetivo:** Obtener reportes administrativos y operativos.

**Precondición:** Información de operaciones registrada.

**Flujo principal:**

1. El administrador selecciona el tipo de reporte.
2. Elige ocupación por salida, ventas por vendedor, ingresos por tour o cancelaciones.
3. El sistema procesa la información de todas las operaciones registradas.
4. Muestra el reporte por consola.

**Flujos alternativos:**

• Si no existen datos, se presenta un reporte vacío con una aclaración.

**Postcondición:** Reporte generado y disponible para consulta.

**Trazabilidad:** RF24 · RN04–RN07, RN20–RN27 · S07.

## 2.4. Matriz de trazabilidad

La siguiente matriz vincula cada caso de uso con su requerimiento funcional principal y el actor responsable.

| CU | Caso de uso | RF / Supuesto | Actor |
|---|---|---|---|
| CU01 | Administrar tours | RF01 | Administrador |
| CU02 | Gestionar itinerarios | RF02 | Administrador |
| CU03 | Gestionar tarifas | RF03 | Administrador |
| CU04 | Programar salidas | RF04 | Coordinador |
| CU05 | Gestionar estados de salidas | RF05 | Coordinador |
| CU06 | Registrar clientes | RF06 | Vendedor |
| CU07 | Registrar pasajeros | RF07 | Vendedor |
| CU08 | Crear reserva | RF08 | Vendedor |
| CU09 | Calcular precio de reserva | RF09 | Vendedor |
| CU10 | Evaluar promociones | RF10 | Vendedor |
| CU11 | Gestionar servicios adicionales | RF11 | Vendedor |
| CU12 | Registrar pagos | RF12 | Vendedor |
| CU13 | Controlar saldos | RF13 | Vendedor |
| CU14 | Confirmar o anular reservas | RF14 | Vendedor |
| CU15 | Cancelar reserva | RF15 | Vendedor |
| CU16 | Modificar reserva | RF16 | Vendedor |
| CU17 | Registrar ajustes económicos | RF17 | Vendedor |

| CU | Caso de uso | RF / Supuesto | Actor |
|---|---|---|---|
| CU18 | Administrar guías y vehículos | RF18 | Coordinador |
| CU19 | Asignar recursos | RF19 | Coordinador |
| CU20 | Emitir lista de pasajeros | RF20 | Coordinador |
| CU21 | Iniciar y finalizar salidas | RF21 | Coordinador |
| CU22 | Liquidar honorarios | RF22 | Coordinador |
| CU23 | Administrar proveedores | RF23 | Administrador |
| CU24 | Generar reportes | RF24 | Administrador |

Las reglas de negocio y los supuestos específicos se encuentran referenciados en las fichas textuales del apartado 2.3.

## 2.5. Aclaraciones del modelado

### *Diferencia entre salida confirmada y reserva confirmada*

Una salida se confirma cuando alcanza el cupo mínimo establecido, de acuerdo con RN05.

Una reserva se confirma cuando cumple las condiciones de pago establecidas en RN20 y S04.

Ambas confirmaciones corresponden a procesos y estados diferentes.

### *Ocupación de cupos*

Todos los pasajeros ocupan un lugar, incluidos los menores de tres años que no abonan tarifa.

Las reservas pendientes pueden retener cupos temporalmente, de acuerdo con S04 y S09.

### *Reservas próximas a la salida*

Cuando una reserva se realiza dentro de los diez días anteriores al inicio de la salida, se exige el pago total durante el registro.

En ese caso, no se genera una reserva pendiente con cupos retenidos sin el pago correspondiente.

### *Cálculo de precios*

El cálculo considera el siguiente orden:

1. Tarifa de temporada.
2. Tarifa correspondiente al tipo de pasajero.
3. Aplicación de una única promoción comercial.
4. Incorporación de servicios adicionales.
5. Recargo por financiación con tarjeta en cuotas, cuando corresponda.

Este criterio responde a RN12–RN22 y a los supuestos S05, S06 y S11.

### *Criterios UML*

Los actores se relacionan con los casos de uso mediante asociaciones.

Las relaciones «include» se utilizan para comportamientos obligatorios modelados como casos de uso independientes.

Los flujos alternativos y las excepciones se describen en las fichas textuales.

Las relaciones «extend» se utilizan solo para los comportamientos condicionales que figuran en las fichas textuales (por ejemplo, CU11 respecto de CU08).

La organización de los diagramas por módulos responde a criterios de claridad visual y no representa sistemas independientes.

## 2.6. Conclusión

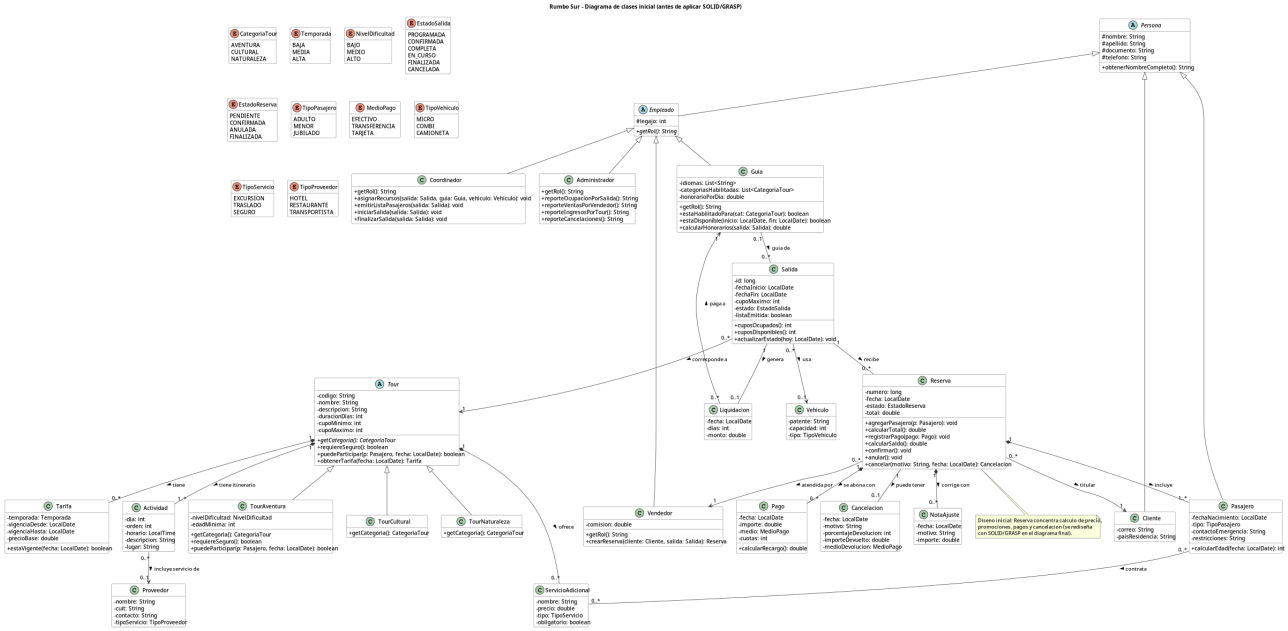
El modelado de casos de uso permite identificar las funcionalidades del Sistema de Gestión Rumbo Sur, las responsabilidades de sus actores y las principales condiciones que deben respetarse durante su funcionamiento.

Los 24 casos de uso documentados mantienen correspondencia con los 24 requerimientos funcionales (RF01–RF24).

Este análisis constituye una base para elaborar los diagramas de clases y de secuencia y para desarrollar posteriormente la aplicación en Java, manteniendo la coherencia entre requerimientos, diseño e implementación.

# 6. Diagrama de clases inicial

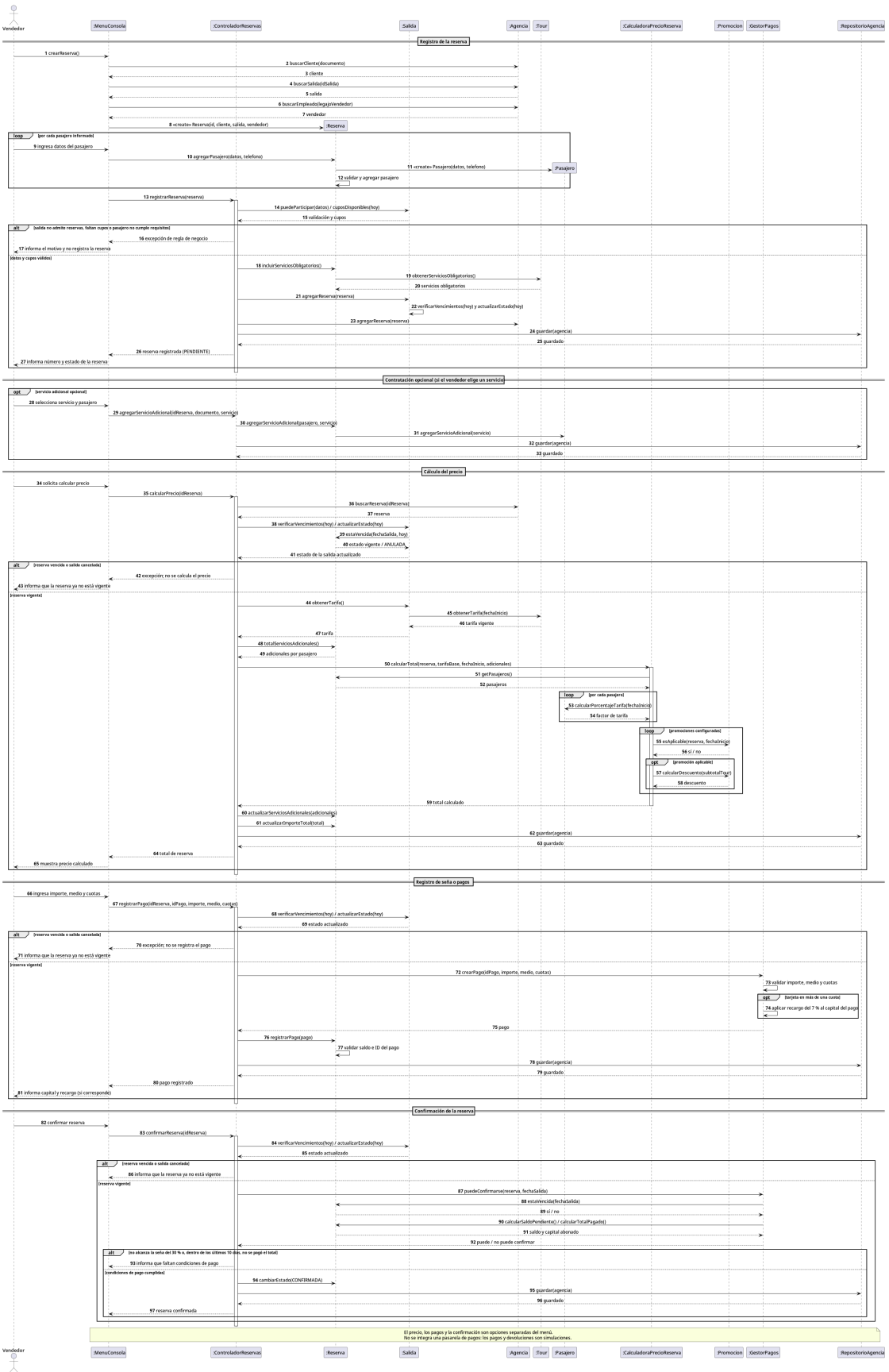
El siguiente diagrama representa el modelo inicial de clases, previo a la aplicación de los principios SOLID y patrones GRASP.



# 7. Diagramas de secuencia

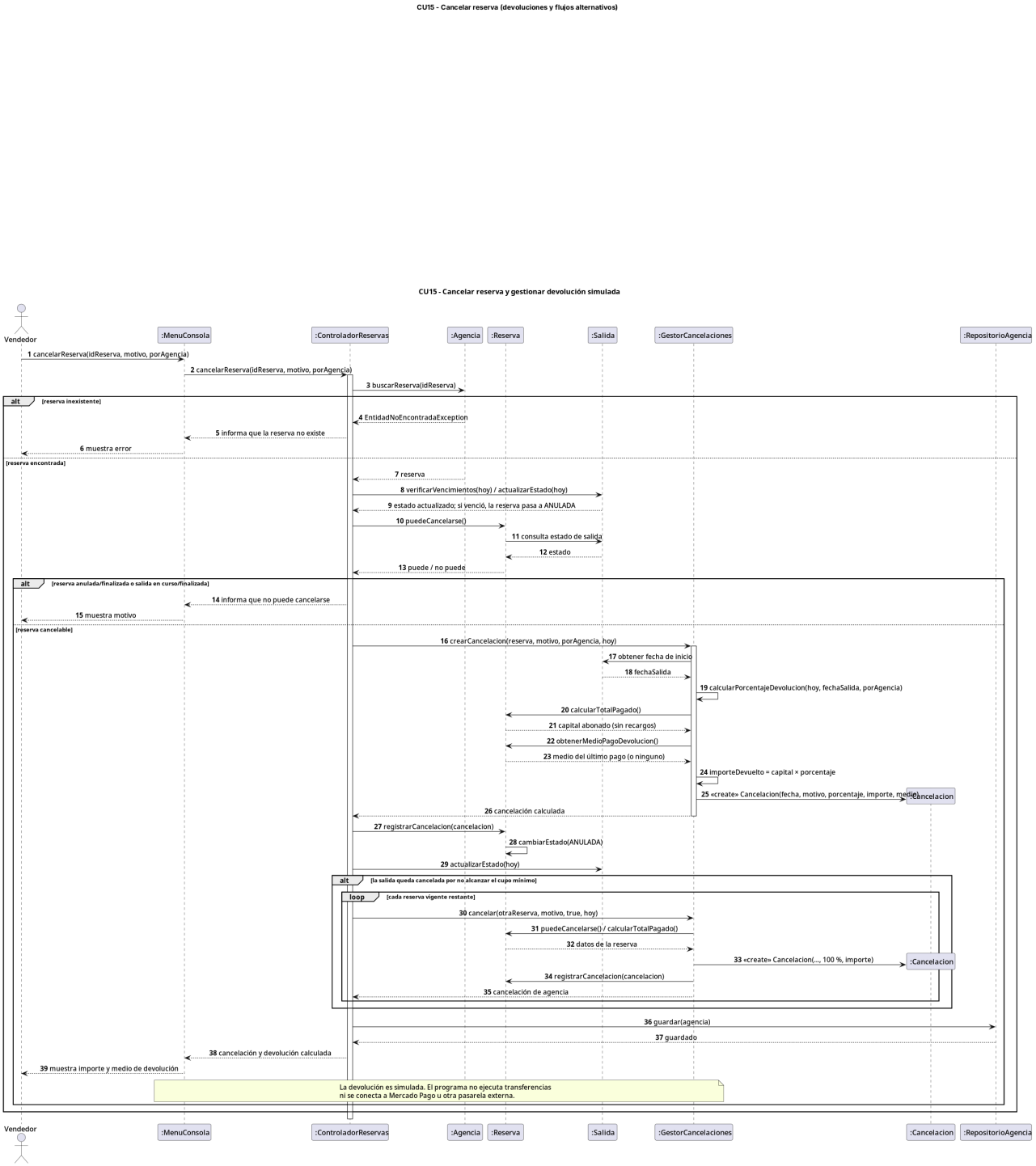
## 7.1. Crear reserva (CU08)

El diagrama representa el flujo de creación de una reserva, incluyendo validaciones y caminos alternativos definidos en el caso de uso.



## 7.2. Cancelar reserva (CU15)

El diagrama representa la consulta de la reserva, la validación de las condiciones de cancelación, el cálculo de la devolución y la actualización de los registros.



# 8. Aplicación de principios SOLID y patrones GRASP

# 5. Aplicación de principios SOLID y patrones GRASP

La aplicación de los principios SOLID y los patrones GRASP tiene como objetivo establecer una distribución adecuada de responsabilidades entre las clases del Sistema de Gestión Rumbo Sur, favoreciendo la cohesión, el bajo acoplamiento, la extensibilidad y el mantenimiento del software.

Las decisiones propuestas se fundamentan en los requerimientos funcionales RF01–RF24, las reglas de negocio RN01–RN33, los supuestos S01–S22 y los casos de uso previamente identificados.

Dado que el proyecto se encuentra en la etapa de diseño, los problemas descriptos representan riesgos potenciales de implementación y no defectos comprobados en código existente.

Las clases propuestas se contrastan con el diagrama de clases final para mantener la coherencia entre los artefactos del proyecto.

## 5.1. Problemas detectados

A partir del análisis de los requerimientos funcionales, las reglas de negocio y los casos de uso definidos para Rumbo Sur, se identificaron posibles problemas de diseño relacionados con la distribución de responsabilidades, la dependencia entre clases y la incorporación de nuevas funcionalidades.

Debido a que el sistema contempla procesos comerciales, administrativos y operativos, resulta necesario evitar que una misma clase concentre operaciones que deberían estar separadas.

Los problemas identificados se presentan como situaciones que podrían surgir durante el diseño y la implementación. Por lo tanto, se proponen decisiones preventivas basadas en SOLID y GRASP.

### *P01 – Concentración de responsabilidades en la gestión de reservas*

La creación de una reserva involucra verificar cupos, registrar pasajeros, calcular importes, aplicar promociones y controlar pagos.

Si todas estas operaciones se implementaran directamente en una única clase `Reserva`, esta tendría múltiples motivos de cambio, dificultando su mantenimiento.

### *P02 – Dependencia de condiciones específicas para calcular descuentos*

El sistema contempla tarifas según el tipo de pasajero y promociones por reserva anticipada o por grupo.

Implementar todas las variantes mediante condiciones dentro de un único método dificultaría incorporar nuevas promociones y aumentaría el riesgo de modificar cálculos existentes.

### *P03 – Acoplamiento entre la lógica de negocio y la persistencia*

El sistema deberá conservar información mediante archivos, según el supuesto S07.

Si las clases encargadas de gestionar reservas también realizaran directamente la lectura y escritura de archivos, cualquier modificación del almacenamiento afectaría la lógica de negocio.

### *P04 – Distribución inadecuada de las responsabilidades de cálculo*

El sistema necesita calcular importes, saldos y cupos disponibles.

Si estas operaciones se concentraran en un controlador general, este debería acceder continuamente a los datos internos de diferentes objetos, incrementando sus dependencias.

### *P05 – Falta de separación entre la interfaz y los procesos del sistema*

Los casos de uso de creación y cancelación de reservas requieren varias validaciones y operaciones coordinadas.

Si la interfaz de usuario ejecutara directamente esas reglas, se dificultaría su reutilización y la realización de pruebas independientes.

### *P06 – Comportamientos sin un lugar natural en las clases del negocio*

El cálculo del total de una reserva necesita datos de la salida, de los pasajeros, de las tarifas y de las promociones. A su vez, la política de devolución depende de los días restantes hasta la salida y de los pagos realizados.

Ninguna clase del dominio (`Reserva`, `Salida` o `Pasajero`) representa por sí sola todo ese conocimiento.

Si estas responsabilidades se asignaran completamente a `Reserva`, la clase tendría que conocer reglas de tarifas, promociones y devoluciones, perdiendo cohesión y aumentando su acoplamiento.

---

## 5.2. Tres principios SOLID

Se seleccionaron tres principios SOLID especialmente relevantes para el diseño de Rumbo Sur:

- **SRP – Principio de Responsabilidad Única.**
- **OCP – Principio Abierto/Cerrado.**
- **DIP – Principio de Inversión de Dependencias.**

### *5.2.1. SRP – Principio de Responsabilidad Única*

**Problema identificado:** P01 – Concentración de responsabilidades en la gestión de reservas.

#### *Fundamentación*

La gestión de reservas comprende actividades que responden a reglas de negocio diferentes.

Por ejemplo, el cálculo del importe depende de las tarifas y promociones, mientras que el registro de pagos depende de las condiciones de cobro y confirmación.

Si una misma clase concentrara todos estos procesos, una modificación de las promociones podría afectar accidentalmente las operaciones de pagos o cancelaciones.

El principio de Responsabilidad Única establece que una clase debe tener un motivo principal de cambio, relacionado con la responsabilidad que cumple dentro del sistema.

#### *Decisión de diseño*

Se propone distribuir las responsabilidades entre clases especializadas:

- `Reserva`**:** mantiene los datos de la reserva, sus pasajeros, servicios adicionales, estado, importe total, vencimiento de pago y pagos asociados.
- `CalculadoraPrecioReserva`**:** calcula el importe correspondiente al tour, las promociones y los servicios adicionales.
- `GestorPagos`**:** coordina el registro de pagos y aplica las condiciones económicas correspondientes al medio de pago.
- `GestorCancelaciones`**:** aplica la política de cancelación y determina los importes de devolución.

La clase `Reserva` conserva las operaciones propias de su estado, como calcular el saldo pendiente, registrar los pagos asociados, confirmar o anular la reserva y verificar su vencimiento.

Las clases gestoras coordinan procesos especializados que requieren consultar o modificar varios objetos.

#### *Clases involucradas*

`Reserva`, `CalculadoraPrecioReserva`, `GestorPagos` y `GestorCancelaciones`.

#### *Ventaja obtenida*

Esta separación mejora la cohesión, facilita las pruebas unitarias y permite modificar las políticas de precios, pagos o cancelaciones sin alterar innecesariamente otras responsabilidades.

**Trazabilidad:** RF08–RF17; RN10–RN28 y RN33; S04–S06.

---

### *5.2.2. OCP – Principio Abierto/Cerrado*

**Problema identificado:** P02 – Dependencia de condiciones específicas para calcular descuentos.

#### *Fundamentación*

Rumbo Sur contempla promociones por reserva anticipada y por grupo.

La primera establece un descuento del 15 % cuando la reserva se realiza con más de 60 días de anticipación; la segunda establece un descuento del 10 % para reservas de seis o más pasajeros.

Según RN17, ambas promociones no son acumulables y debe seleccionarse la más beneficiosa.

Si cada promoción se implementara mediante condiciones dentro de `CalculadoraPrecioReserva`, incorporar una nueva promoción exigiría modificar repetidamente el método principal.

El principio Abierto/Cerrado establece que las clases deben estar abiertas a la extensión, pero cerradas a modificaciones innecesarias de su comportamiento existente.

### *Decisión de diseño*

Se propone aplicar polimorfismo mediante una interfaz común:

- `Promocion`**:** interfaz que define la operación `calcularDescuento(reserva, subtotalTour)`.
- `PromocionAnticipada`**:** implementación que evalúa el descuento por anticipación.
- `PromocionGrupo`**:** implementación que evalúa el descuento por cantidad de pasajeros.
- `CalculadoraPrecioReserva`**:** clase que consulta las promociones disponibles mediante la interfaz común y selecciona el mayor descuento aplicable.

Cada promoción encapsula sus propias condiciones.

La incorporación de nuevas promociones podrá realizarse mediante nuevas implementaciones de `Promocion`, sin modificar el algoritmo central de evaluación, siempre que se mantenga el contrato definido.

### *Orden de cálculo propuesto*

1. Obtener la tarifa base correspondiente a la temporada.
2. Aplicar la tarifa según el tipo de pasajero.
3. Evaluar las promociones comerciales y seleccionar la más beneficiosa, sin acumular descuentos incompatibles.
4. Incorporar los servicios adicionales, incluyendo el seguro obligatorio cuando corresponda.
5. Aplicar, cuando corresponda, el recargo por financiación con tarjeta únicamente sobre el importe financiado.

La tarifa especial para jubilados se considera en el cálculo según las reglas de negocio definidas para el tipo de pasajero.

Este orden deberá mantenerse consistente con las reglas de negocio y con la implementación definitiva.

### *Clases involucradas*

`Promocion`, `PromocionAnticipada`, `PromocionGrupo` y `CalculadoraPrecioReserva`.

### *Ventaja obtenida*

El diseño permite incorporar nuevas promociones mediante implementaciones adicionales de la interfaz, sin modificar el algoritmo central de cálculo.

También facilita probar cada promoción de manera independiente.

**Trazabilidad:** RF09–RF11; RN12–RN19 y RN22; S03, S05, S06 y S11.

---

## *5.2.3. DIP – Principio de Inversión de Dependencias*

**Problema identificado:** P03 – Acoplamiento entre la lógica de negocio y la persistencia.

### *Fundamentación*

El supuesto S07 establece que la información del sistema se conservará mediante archivos.

Sin embargo, si las operaciones de reserva dependieran directamente de una clase encargada de escribir archivos, la lógica de negocio quedaría vinculada a un mecanismo particular de almacenamiento.

Esto dificultaría modificar la persistencia y realizar pruebas independientes.

El principio de Inversión de Dependencias establece que los componentes de alto nivel deben depender de abstracciones, en lugar de depender directamente de implementaciones concretas.

### *Decisión de diseño*

Se propone establecer una abstracción para el acceso a los datos:

- `RepositorioAgencia`**:** interfaz que define las operaciones `guardar(agencia)` y `cargar()`.

- `RepositorioAgenciaArchivo`**:** clase que implementa la interfaz y realiza la lectura y escritura del archivo de datos (serialización de objetos Java, S07).
- `ControladorReservas` **y** `ControladorOperaciones`**:** clases que usan la abstracción `RepositorioAgencia`, sin depender directamente de los detalles del almacenamiento.
- `Agencia`**:** objeto raíz que mantiene en memoria las colecciones del sistema (tours, salidas, clientes, reservas, empleados, vehículos y proveedores) y ofrece las búsquedas. El repositorio guarda y recupera la agencia completa, por lo que una sola interfaz cubre todas las entidades.

La implementación concreta del repositorio se proporcionará al construir o configurar el controlador.

De esta forma, los componentes de coordinación dependen de una interfaz y no de una tecnología específica.

### *Clases involucradas*

`ControladorReservas`, `ControladorOperaciones`, `RepositorioAgencia`, `RepositorioAgenciaArchivo` y `Agencia`.

Al guardar la agencia completa, el mismo criterio vale para todas las entidades del sistema (S07).

### *Ventaja obtenida*

Esta decisión permite modificar el mecanismo de persistencia sin alterar la coordinación de reservas.

También facilita reemplazar el repositorio por una implementación de prueba y verificar las operaciones sin utilizar archivos reales.

**Trazabilidad:** RF08, RF12–RF17; S02 y S07.

---

## 5.3. Cuatro patrones GRASP

Se seleccionaron cuatro patrones GRASP para orientar la asignación de responsabilidades entre las clases del sistema:

- **Experto en Información (Information Expert).**
- **Creador (Creator).**
- **Controlador (Controller).**
- **Fabricación Pura (Pure Fabrication).**

### *5.3.1. Experto en Información (Information Expert)*

**Problema identificado:** P04 – Distribución inadecuada de las responsabilidades de cálculo.

### *Fundamentación*

Rumbo Sur necesita calcular importes, saldos pendientes y cupos disponibles.

Si todas estas operaciones se concentraran en un controlador general, este debería consultar permanentemente información interna de diferentes objetos, incrementando sus dependencias.

El patrón Experto en Información establece que una responsabilidad debe asignarse, preferentemente, a la clase que dispone de los datos necesarios para cumplirla.

### *Decisión de diseño*

Se propone distribuir las responsabilidades de la siguiente manera:

- `Salida`**:** conoce su capacidad y la lista de sus reservas (asociación `Salida`–`Reserva` en el diagrama), por lo que puede determinar los cupos disponibles. También conoce su fecha de inicio, por lo que responde cuántos días faltan (`diasHastaInicio`) y si corresponde exigir el pago total (`exigePagoTotal`, faltan 10 días o menos). Además es quien anula las reservas vencidas de su lista (`verificarVencimientos`) y quien finaliza sus reservas cuando termina (`finalizar`).
- Pasajero: conoce su fecha de nacimiento y su tipo, por lo que calcula su factor de tarifa (calcularPorcentajeTarifa).
- `Guia`**:** conoce sus salidas asignadas y por eso responde si está disponible en un rango de fechas.
- `Reserva`**:** conoce el importe total y los pagos asociados, por lo que puede calcular el saldo pendiente.

- `CalculadoraPrecioReserva`**:** reúne las reglas necesarias para determinar el precio a partir de tarifas, promociones y servicios adicionales.
- `ControladorReservas`**:** coordina las operaciones, pero delega los cálculos en las clases responsables.

Por ejemplo, cuando el vendedor intenta crear una reserva, `ControladorReservas` solicita a `Salida` que verifique la disponibilidad, en lugar de calcular directamente los cupos.

Para determinar los cupos disponibles, `Salida` debe considerar tanto las reservas confirmadas como las reservas pendientes que mantienen una retención vigente, según RN07, S04 y S09.

Del mismo modo, para consultar el saldo de una reserva, se utiliza el comportamiento correspondiente de `Reserva`, y para saber si hace falta pago total inmediato, el controlador pregunta a `Salida` en lugar de calcular los días por su cuenta.

### *Clases involucradas*

`Salida`, `Reserva`, `CalculadoraPrecioReserva` y `ControladorReservas`.

### *Ventaja obtenida*

Esta distribución favorece la cohesión, evita la concentración de lógica en el controlador y facilita las pruebas unitarias.

Cada clase administra las operaciones relacionadas con la información que conoce.

**Trazabilidad:** RF04, RF05, RF08, RF09 y RF13; RN07; S04 y S09.

---

## *5.3.2. Creador (Creator)*

**Problema identificado:** creación y asociación de pasajeros sin un responsable claramente definido.

### *Fundamentación*

Cada reserva contiene uno o más pasajeros y está asociada a una salida.

La creación e incorporación de pasajeros requiere mantener relaciones consistentes y respetar las restricciones del negocio.

Si el menú o el controlador crearan los pasajeros y modificaran directamente la colección de la reserva, podrían generarse asociaciones inconsistentes.

El patrón Creador establece criterios para asignar la responsabilidad de crear objetos a una clase que los contiene, los agrega o administra su ciclo de vida.

### *Decisión de diseño*

Se propone que la clase `Reserva` sea responsable de crear e incorporar las instancias de `Pasajero` que componen su registro, mediante la operación:

```
agregarPasajero(datos: DatosPasajero)
```

El `ControladorReservas` coordinará la creación de la reserva y verificará previamente la disponibilidad y las restricciones del tour.

Una vez superadas esas validaciones, `Reserva` administrará la creación y asociación de sus pasajeros.

Esta decisión se fundamenta en que `Reserva` agrupa y administra a los pasajeros vinculados con una operación comercial. Por el mismo criterio, `Reserva` crea sus `Pago` (`registrarPago`, después de que `GestorPagos` calcula el recargo) y sus `NotaAjuste` (`registrarAjuste`).

La propia `Reserva` es creada por `ControladorReservas`, porque es la clase que recibe los datos iniciales (cliente, salida, vendedor y número de reserva): el patrón Creador también admite como creador a quien dispone de los datos de inicialización.

En el diagrama de clases final, esta relación se representa mediante composición:

```
Reserva "1" -- "1.." Pasajero
```

### *Clases involucradas*

`Reserva`, `Pasajero` y `ControladorReservas`.

### *Ventaja obtenida*

Se establece un criterio claro de creación y asociación de objetos, evitando que diferentes componentes modifiquen directamente la colección de pasajeros.

Esto favorece la consistencia de las reservas y permite centralizar las validaciones propias de su composición.

**Trazabilidad:** RF07 y RF08; RN10, RN11 y RN19; S03 y S04.

---

## *5.3.3. Controlador (Controller)*

**Problema identificado:** P05 – Falta de separación entre la interfaz y los procesos del sistema.

### *Fundamentación*

El caso de uso **CU08 – Crear reserva** requiere coordinar el registro de pasajeros, la validación de disponibilidad, el cálculo del importe, la incorporación de servicios adicionales y las condiciones de pago.

Asimismo, **CU15 – Cancelar reserva** requiere verificar condiciones, calcular devoluciones, actualizar el estado y liberar los cupos correspondientes.

Estas operaciones no deberían ejecutarse directamente desde el menú o la interfaz de usuario.

El patrón Controlador establece que los eventos del sistema deben recibirse y coordinarse mediante una clase responsable de delegar las operaciones a los objetos adecuados.

### *Decisión de diseño*

Se proponen dos controladores de casos de uso. `ControladorReservas` recibe las solicitudes del módulo comercial (clientes, reservas, pagos, cancelaciones, reemplazos y ajustes: CU06 a CU17). `ControladorOperaciones` recibe las del módulo operativo y de catálogo (tours, tarifas, proveedores, guías, vehículos, salidas, asignación de recursos, lista de pasajeros, inicio y fin de salida y liquidación: CU01 a CU05, CU18 a CU23). Los reportes (CU24) los resuelve `GeneradorReportes`. `MenuConsola` solo muestra opciones y delega en ellos.

Este controlador delegará las operaciones en las clases especializadas:

- `Salida`**:** consulta de disponibilidad y condiciones de participación.
- `Reserva`**:** administración de pasajeros, servicios, estado, vencimiento y pagos asociados.
- `CalculadoraPrecioReserva`**:** determinación de importes.
- `GestorPagos`**:** registro de pagos y aplicación de condiciones de cobro.
- `GestorCancelaciones`**:** aplicación de la política de devolución.
- `Agencia` **y** `RepositorioAgencia`**:** búsqueda de entidades y almacenamiento de los datos.

El controlador coordinará el flujo de trabajo sin concentrar los cálculos específicos de cada regla de negocio.

En el código, MenuConsola reúne los datos, construye la Reserva y agrega sus pasajeros mediante Reserva.agregarPasajero. Luego, ControladorReservas.registrarReserva valida el estado de la salida, los cupos, los requisitos y los servicios obligatorios antes de registrarla. El cálculo del precio, los adicionales opcionales, el pago y la confirmación se ejecutan mediante operaciones separadas.

Para las reservas pendientes, también coordinará la verificación del vencimiento, según S04 y S09.

Ante una solicitud de cancelación, recuperará la reserva, verificará si puede cancelarse, solicitará el cálculo de devolución a `GestorCancelaciones` y coordinará la actualización de su estado.

### *Clases involucradas*

`ControladorReservas`, `Reserva`, `Salida`, `CalculadoraPrecioReserva`, `GestorPagos`, `GestorCancelaciones`, `Agencia` y `RepositorioAgencia`.

### *Ventaja obtenida*

La separación permite mantener la interfaz independiente de la lógica de negocio, reutilizar las operaciones desde diferentes puntos del sistema y facilitar la construcción de los diagramas de secuencia.

**Trazabilidad:** CU08 y CU15; RF08–RF17; RN07, RN10–RN28; S04, S06 y S09.

---

### 5.3.4. Fabricación Pura (Pure Fabrication)

**Problema identificado:** P06 – Comportamientos sin un lugar natural en las clases del negocio.

#### Fundamentación

El cálculo del precio y la política de devolución no son responsabilidades que puedan asignarse completamente a una reserva, un tour o un pasajero por sí mismos.

Son procesos que combinan información de varias clases y aplican políticas específicas.

Ubicarlos íntegramente en `Reserva` la obligaría a conocer tarifas, promociones, reglas de cobro y políticas de cancelación, reduciendo su cohesión.

El patrón Fabricación Pura propone crear clases que no representan directamente una entidad del negocio, sino que se incorporan al diseño para agrupar comportamientos especializados con alta cohesión y bajo acoplamiento.

#### Decisión de diseño

Se crean las siguientes clases especializadas:

- `CalculadoraPrecioReserva`**:** calcula el total a partir de la tarifa de la salida, el factor correspondiente a cada pasajero, las promociones y los servicios adicionales.
- `GestorPagos`**:** calcula el recargo correspondiente cuando se utiliza financiación con tarjeta y delega en `Reserva` el registro del pago.
- `GestorCancelaciones`**:** aplica la política de cancelación, calcula el porcentaje de devolución y determina el importe a reintegrar.

La clase `Reserva` conserva sus datos y las operaciones sobre su propio estado, como `totalAbonado()`, `calcularSaldo()`, `confirmar()` y `anular()`.

La ejecución efectiva de un reintegro no se considera completada únicamente por calcular el importe de devolución. Ese proceso deberá registrarse según el mecanismo de pagos definido en la implementación.

Esta decisión se diferencia del principio SRP:

- **SRP:** justifica separar responsabilidades según sus motivos de cambio.
- **Fabricación Pura:** justifica crear clases especializadas que no representan directamente una entidad del dominio, pero permiten ubicar comportamientos que no pertenecen naturalmente a una única clase existente.

#### Clases involucradas

`CalculadoraPrecioReserva`, `GestorPagos`, `GestorCancelaciones` y `Reserva`.

#### Ventaja obtenida

La clase `Reserva` mantiene una mayor cohesión, las políticas de cálculo pueden probarse de forma aislada y las reglas de precio, cobro y devolución pueden cambiar sin modificar innecesariamente otras partes del modelo.

**Trazabilidad:** RF09, RF10, RF12 y RF15; RN15, RN16 y RN23–RN28; S10 y S11.

---

## 5.4. Relación entre problemas, SOLID y GRASP

La siguiente tabla resume cómo las decisiones de diseño propuestas permiten abordar los problemas identificados.

| Problema | Descripción | Principio o patrón aplicado | Solución propuesta |
|---|---|---|---|
| P01 | Concentración de responsabilidades en reservas | SOLID: SRP | Separar las responsabilidades de reserva, cálculo de precios, pagos y cancelaciones. |
| P02 | Dificultad para incorporar promociones | SOLID: OCP | Definir una interfaz `Promocion` y distintas implementaciones. |
| P03 | Acoplamiento entre negocio y persistencia | SOLID: DIP | Utilizar `RepositorioAgencia` como abstracción de almacenamiento. |
| P04 | Distribución inadecuada de cálculos | GRASP: Experto en Información | Asignar el cálculo de cupos a `Salida` y el de saldos a `Reserva`. |

| Problema | Descripción | Principio o patrón aplicado | Solución propuesta |
|---|---|---|---|
| P05 | Interfaz acoplada a procesos de negocio | GRASP: Controlador | Incorporar `ControladorReservas` para coordinar los casos de uso. |
| P06 | Cálculos y políticas sin lugar natural en el dominio | GRASP: Fabricación Pura | Crear `CalculadoraPrecioReserva`, `GestorPagos` y `GestorCancelaciones`. |

El patrón **GRASP Creador** complementa estas decisiones al establecer que `Reserva` administra la creación e incorporación de sus pasajeros, manteniendo consistente la relación entre ambos objetos.

## 5.5. Conclusión

La aplicación de los principios SOLID y los patrones GRASP permite establecer criterios concretos para distribuir las responsabilidades del Sistema de Gestión Rumbo Sur.

Los principios **SRP, OCP y DIP** contribuyen a separar funcionalidades, facilitar la incorporación de promociones y evitar dependencias directas con los mecanismos de persistencia.

Por su parte, los patrones **Experto en Información, Creador, Controlador y Fabricación Pura** permiten asignar responsabilidades a las clases adecuadas, coordinar los procesos del sistema y ubicar los cálculos en clases especializadas.

Las decisiones propuestas se relacionan con los requerimientos, las reglas de negocio, los supuestos y los casos de uso definidos para la agencia.

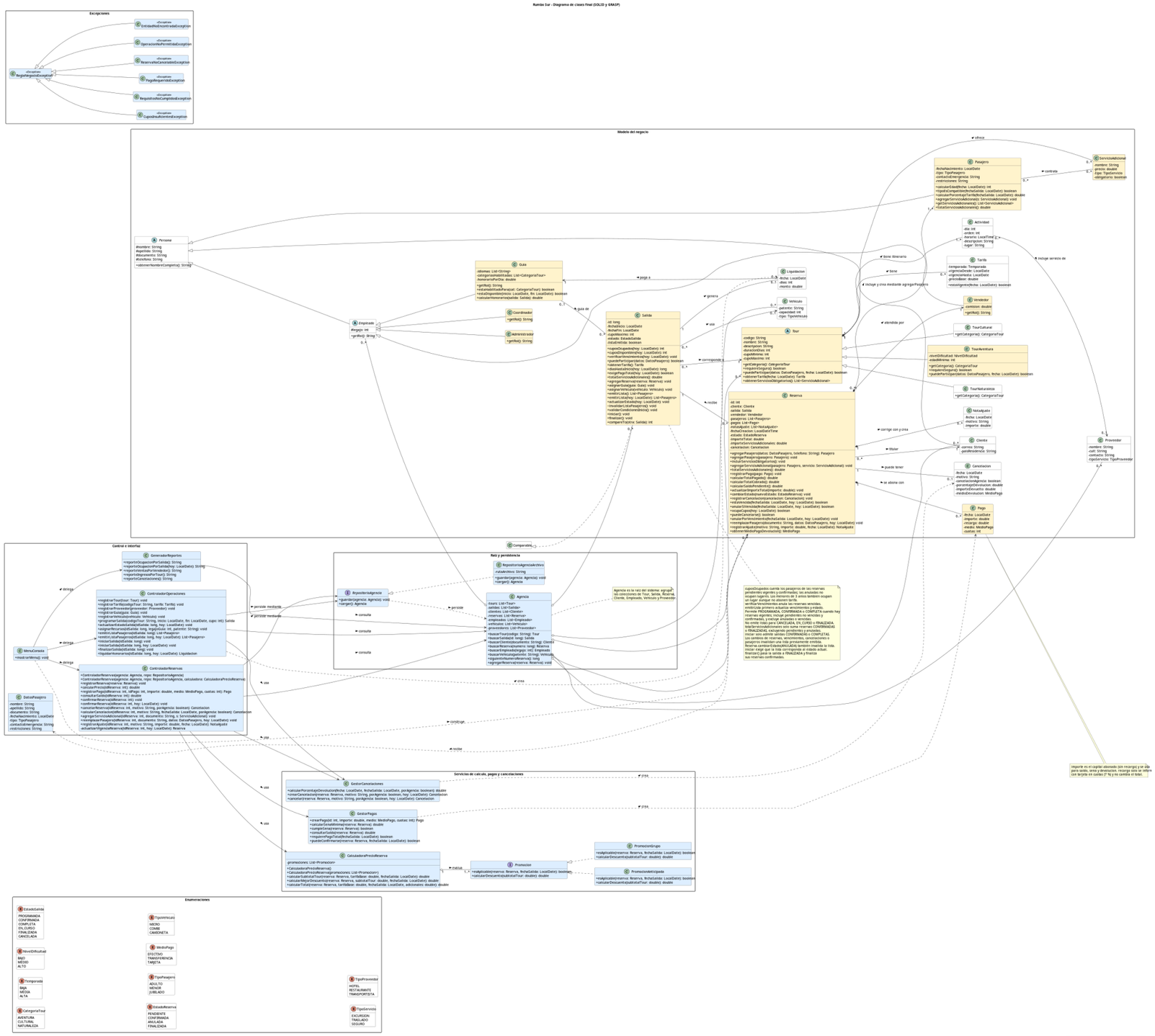
El diseño contempla especialmente la gestión de reservas pendientes, la disponibilidad de cupos, los servicios adicionales, las promociones, los pagos y las cancelaciones.

Las decisiones quedaron reflejadas en el diagrama de clases final, los diagramas de secuencia y la implementación Java. Su trazabilidad entre las dos etapas se resume en el anexo de actualización incorporado al final del documento.

De esta forma, el proyecto establece una base de diseño orientado a objetos que favorece la cohesión, la extensibilidad, la reutilización y el mantenimiento del sistema.

# 9. Diagrama de clases final

Diagrama actualizado a partir de las fuentes PlantUML vigentes tras la integración de Etapa 2. Representa las clases, responsabilidades, relaciones y validaciones que se describen en el anexo de actualización.



# 9.1. Cambios y justificación

El diagrama final incorpora las decisiones de diseño de Etapa 1 y refleja las responsabilidades que quedaron implementadas en Java. Los cambios principales se resumen a continuación.

| Decisión | Aplicación y justificación |
|---|---|
| SRP y Fabricación Pura | Se separan cálculo de precios, pagos y cancelaciones en CalculadoraPrecioReserva, GestorPagos y GestorCancelaciones. Reserva conserva los datos y operaciones sobre su propio estado. |
| OCP – Promociones | CalculadoraPrecioReserva recibe una List<Promocion> por constructor y evalúa cada implementación mediante la interfaz. Se puede incorporar una nueva promoción sin modificar el algoritmo central. |
| DIP – Persistencia | Agencia agrupa las colecciones del sistema y la lógica utiliza la abstracción RepositorioAgencia; RepositorioAgenciaArchivo implementa el guardado y la carga por archivo. |
| GRASP Controlador | MenuConsola presenta las opciones y delega en ControladorReservas, ControladorOperaciones y GeneradorReportes. Los controladores coordinan, sin concentrar todos los cálculos. |
| GRASP Creador | Reserva.agregarPasajero(DatosPasajero, String) crea e incorpora el pasajero. Reserva también crea las notas de ajuste; GestorPagos crea los pagos. |
| GRASP Experto | Salida conoce sus reservas, cupos y fechas; Reserva administra pasajeros, pagos y saldo; Pasajero conoce los datos necesarios para determinar su tarifa. |
| Modelo y relaciones | Los servicios adicionales pertenecen a un Tour y se contratan por pasajero. Salida implementa Comparable para el orden natural por fecha; los reportes también aplican Comparator. |
| Excepciones | Las excepciones propias del dominio se organizan a partir de ReglaNegocioException y representan errores de reglas de negocio, como cupos, requisitos, pagos y operaciones no permitidas. |

*Las firmas completas y las relaciones entre estas clases se encuentran en el diagrama de clases final actualizado de la página anterior.*

# 10. Actualización del diseño durante Etapa 2

Durante la codificación y las pruebas se ajustaron algunas responsabilidades y reglas para que el modelo UML coincidiera con la implementación. Estos cambios se documentan aquí y en docs/uml/05-cambios-etapa-2.md.

| Área / trazabilidad | Cambio implementado | Motivo |
|---|---|---|
| Vencimientos<br>S04, S09, RN21 | Se anulan las reservas pendientes al vencer su retención y las confirmadas que conservan saldo impago al superar el límite de pago. Los cupos quedan liberados. | Aplicar la regla antes de consultar disponibilidad o realizar operaciones sobre una reserva. |
| Alta de reservas<br>CU08, RF08 | El menú reúne los datos y crea la reserva. El controlador valida el estado de la salida, cupos, requisitos del pasajero y servicios obligatorios antes de registrarla. | Evitar reservas en salidas canceladas, en curso o finalizadas y mantener las validaciones fuera de la interfaz. |
| Lista de pasajeros<br>CU20, RN29 | La lista incluye reservas confirmadas y pendientes con retención vigente; excluye anuladas o vencidas. Puede emitirse como lista provisional mientras la salida siga en estado compatible. | Interpretar «reservas vigentes» junto con la retención temporal definida por S04/S09. |
| Integridad de la lista<br>RN29, RN30 | Los cambios de pasajeros, reservas, vencimientos o cancelaciones invalidan una lista anterior. Debe emitirse nuevamente antes de iniciar. | Impedir que se inicie una salida con una lista que ya no representa sus reservas vigentes. |
| Cancelaciones<br>S15, CU15 | Una reserva solo puede cancelarse si está pendiente o confirmada y su salida no está en curso ni finalizada. La validación también se aplica en el objeto y el gestor. | Hacer cumplir la misma regla desde el controlador y desde las operaciones del dominio. |
| Honorarios<br>RN31, RN32 | La base de ventas de adicionales considera reservas confirmadas o finalizadas. Se excluyen pendientes y anuladas. | Evitar liquidar honorarios sobre ventas que no quedaron confirmadas o fueron anuladas. |
| Pagos y devoluciones<br>RN22, RN27, S11, S13, S17 | El capital y el recargo del 7 % se conservan por separado. La devolución se calcula sobre el capital y registra el medio de devolución conforme al supuesto adoptado. | Mantener el saldo y los porcentajes de devolución independientes del recargo de financiación. La devolución es simulada. |

*Condición para iniciar una salida: además de la lista vigente, el sistema exige que la salida esté confirmada o completa, que haya guía habilitado y que el vehículo cumpla la capacidad requerida.*

# 10.1. Correspondencia entre diagramas de secuencia y código

Los dos diagramas de secuencia se actualizaron desde sus fuentes PlantUML para representar las operaciones que ejecuta la versión Java final. Ambos conservan flujos alternativos y condiciones de error.

## CU08 – Crear, calcular y confirmar una reserva

El diagrama 02-secuencia-crear-reserva separa los momentos del proceso, tal como ocurre en la consola:

- Registro: MenuConsola consulta cliente, salida y vendedor, construye Reserva y agrega cada pasajero mediante Reserva.agregarPasajero(DatosPasajero, String). Después llama a ControladorReservas.registrarReserva.
- Validación: el controlador verifica vigencia de la salida, disponibilidad, requisitos y servicios obligatorios. Los errores por cupos insuficientes o requisitos no cumplidos se representan como caminos alternativos.
- Adicionales: los servicios opcionales se incorporan por pasajero cuando el vendedor los selecciona; los obligatorios se agregan automáticamente según el tour.
- Precio: calcularPrecio consulta la tarifa vigente, calcula el subtotal por pasajero y aplica la promoción más favorable mediante la interfaz Promocion.
- Pago y confirmación: el registro del pago y la confirmación son opciones separadas. Se valida la seña del 30 % y, cuando corresponde por proximidad de la salida, el pago total.

## CU15 – Cancelar reserva y gestionar devolución simulada

El diagrama 03-secuencia-cancelar-reserva representa la consulta de la reserva, la actualización de vencimientos, la verificación de puedeCancelarse(), el cálculo del porcentaje por GestorCancelaciones y el registro de la cancelación. Si la agencia cancela una salida por no alcanzar el cupo mínimo, se recorren las reservas vigentes y se registra la devolución del 100 % en cada caso.

*Alcance de los pagos: el programa registra pagos y devoluciones en el sistema y muestra mensajes por consola. No se conecta a Mercado Pago, bancos ni pasarelas externas; las devoluciones se simulan.*

*Fuentes vigentes: docs/uml/02-secuencia-crear-reserva.puml, docs/uml/03-secuencia-cancelar-reserva.puml y sus exportaciones SVG.*

# 10.2. Verificación técnica y pruebas

La implementación fue compilada con compatibilidad Java 17 mediante javac --release 17. Se ejecutaron las seis clases de prueba incluidas en test/; todas finalizaron correctamente.

| Suite automatizada | Cobertura principal | Resultado |
|---|---|---|
| AgenciaTest | Búsqueda de tours, clientes y entidades inexistentes. | OK |
| SalidaTest | Cupos, estados, vencimientos, lista vigente, cancelación según estado y honorarios. | OK |
| GeneradorReportesTest | Reportes de ocupación, ventas, ingresos y cancelaciones. | OK |
| ControladorOperacionesTest | Asignación de recursos y cancelación por cupo mínimo. | OK |
| ControladorReservasTest | Alta, precios, pagos, confirmación, cancelación, ajustes y vencimientos. | OK |
| CalculadoraPrecioReservaTest | Aplicación y extensión de promociones mediante la interfaz. | OK |

Resultado global: seis suites correctas y 17 escenarios automatizados documentados en docs/Plan_de_pruebas.md. También se realizaron pruebas de humo del menú y de persistencia por serialización, recuperando dos tours, dos salidas y tres reservas.

Las pruebas automatizadas se implementan como clases Java ejecutables con main y aserciones, sin dependencias externas. No se utiliza JUnit. El plan incluye además diez pruebas manuales (PM01–PM10) recomendadas para la demostración; se presentan como guía y no como pruebas automatizadas ya ejecutadas.

*Antes de cerrar la entrega en GitHub, el grupo debe confirmar que existan los tags etapa-1 y etapa-2, que los commits de los tres integrantes sean verificables y que las pruebas manuales PM01–PM10 hayan sido recorridas en el equipo del grupo.*

*Este anexo complementa el análisis, los requerimientos, los supuestos, los casos de uso, el diagrama de clases inicial, los dos diagramas de secuencia y la fundamentación de los tres principios SOLID y cuatro patrones GRASP incluidos en las secciones anteriores.*